

A Drop-in KEM Replacement for Client Signatures in Post-Quantum SSH

Hongbo Liu^{*†}, Yufan Su[‡], Jiangxia Ge[§], Qionglu Zhang^{*†}, Zhaoxuan Li^{*†},
Xianhui Lu^{*†}, Li Song^{*†}, Wenhua Gao[¶], Li Zhou^{||}

^{*} State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, CAS, Beijing, China

[†] School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

[‡] School of Control and Computer Engineering, North China Electric Power University, Beijing, China

[§] China Telecom Quantum Information Technology Group Co., Ltd., Hefei, China

[¶] Beijing Certificate Authority Co., Ltd. (BJCA), Beijing, China

^{||} Institute of Software, Chinese Academy of Sciences, Beijing, China

Corresponding authors: Qionglu Zhang and Zhaoxuan Li ({zhangqionglu,lizhaoxuan}@iie.ac.cn)

Abstract—The transition to post-quantum cryptography is reshaping the Secure Shell (SSH) protocol for remote administration. Post-quantum key exchange has been deployed in OpenSSH and is being standardized, while SSH authentication largely remains a signature-replacement effort. This path preserves the familiar public-key credential model, but inherits the size and computation overhead of post-quantum signatures, which can increase latency, traffic, and server-side load. KEM-based authentication offers a natural alternative to this signature-centric path, and SSH makes this especially attractive at the user-authentication layer, which is method-extensible, separated from transport-layer key exchange and host-key authentication, and already protected by the established channel.

We present a drop-in KEM-based user-authentication method for SSH that replaces client public-key signatures with a session-bound challenge-response proof. The method fits into SSH’s existing user-authentication framework, preserving the public-key credential model and enabling incremental deployment alongside existing methods. We provide a reduction-based security argument in the post-quantum ACCE framework, implement the design in OpenSSH using liboqs, and evaluate it under representative RTTs, TCP initial-window settings, and post-quantum migration configurations. Our results show that KEM-based authentication is competitive with compact signature-based authentication under representative network settings, while reducing median handshake latency by up to about 10% against large-signature hybrid baselines. The advantages are clearer when post-quantum signatures stress transmission or computation: median latency under small TCP initial windows falls by up to 7.3% versus ML-DSA and 17.9% versus SLH-DSA, while server-side online cryptographic cost is 59.1% lower than that for ML-DSA in the same NIST category.

Index Terms—post-quantum SSH, KEM-based authentication, user authentication, ACCE security.

I. INTRODUCTION

The Secure Shell (SSH) protocol is widely deployed for secure remote administration. Its security relies on public-key

mechanisms to establish the encrypted transport channel, authenticate the server, and commonly authenticate users through public-key credentials inside that channel [1]–[3]. As quantum computing threatens the assumptions underlying these mechanisms, SSH deployments and standards are beginning to migrate toward post-quantum cryptography [4].

This transition has so far been asymmetric. On the key-exchange side, post-quantum KEMs fit the role of establishing fresh shared secrets, and SSH has already begun to adopt them. OpenSSH has deployed transport-layer methods such as `mlkem768x25519-sha256` [5], and related SSH key-exchange mechanisms are moving through standardization [6], [7]. Authentication has followed a more conservative path. Existing prototypes and standardization efforts largely preserve the `publickey` method by replacing classical signatures, such as Ed25519 [8] and RSA [9], with post-quantum signatures. This path is easy to integrate, but it also makes authentication inherit their size and verification profile. Prior measurements show that signature-based post-quantum SSH handshakes can increase latency by up to 50% in the worst case under realistic network conditions, especially when large authentication objects exceed the TCP initial congestion window [10], [11]. Password authentication avoids client-side signatures, but gives up the registered public-key credential model and its public-key proof-of-possession property. SSH therefore lacks a non-signature option with these properties [10].

KEMs provide a natural basis for authentication because encapsulation to a registered public key allows only the corresponding secret-key holder to derive the secret needed for a session-bound response. Signature-free authentication has long appeared in AKE and secure-channel designs [12]–[16], and recent post-quantum work has turned it into practical protocol designs [17], [18]. These designs show that KEM-based authentication can reduce bandwidth and server CPU costs in post-quantum settings, as illustrated by KEMTLS and the AuthKEM IETF draft [18], [19]. However, they typically integrate KEM credentials into the handshake or key schedule, so deployment requires changes to core protocol messages, transcript binding, key derivation, and interoperability mecha-

nisms in existing implementations.

SSH makes this picture less rigid. Its architecture separates user authentication from the transport handshake that establishes the encrypted channel and authenticates the server. By the time user authentication begins, the channel keys and session identifier have already been established, and the authentication exchange runs inside the protected channel [20], [21]. At the same time, this layer is method-extensible and policy-driven [21], and SSH already accommodates challenge–response mechanisms such as `keyboard-interactive` [22]. This combination of separation and extensibility suggests a more local path for bringing KEM-based authentication into SSH within its existing authentication framework.

A. Our Contributions

In this paper, we present a drop-in user-authentication method that adapts KEM-based authentication to post-quantum SSH. We study this adaptation end to end through a post-quantum ACCE analysis, an OpenSSH implementation, and controlled network evaluation. Our contributions are summarized as follows:

Drop-in KEM authentication without extra SSH round trips. We design KEMUAuth to preserve SSH’s public-key credential model while replacing client signatures with a deniable, session-bound proof of KEM secret-key possession. The proof is bound to the SSH session identifier, the challenge ciphertext, and the authentication context, giving it the same session-binding role as the signed request in the existing `publickey` method. KEMUAuth follows the commonly used two-step `publickey` flow with `USERAUTH_PK_OK` without adding SSH round trips. At the user-authentication layer, KEMUAuth allows servers to authorize KEM credentials alongside existing signature keys and to combine them with SSH’s existing multi-method authentication policies.

Post-quantum ACCE security for KEMUAuth. We analyze SSH with KEMUAuth in the post-quantum ACCE framework, building on prior ACCE analyses of SSH [3]. The main new proof obligation is server acceptance of a client authenticated through KEMUAuth, which extends the existing server-authenticated SSH analysis to mutual authentication across the transport and user-authentication layers. We reduce this case to the IND-CCA security of the client’s long-term KEM credential and the PRF security of the session-bound response function. Since KEMUAuth runs inside the established encrypted channel and does not affect transport-layer key derivation, the authentication argument composes with the existing SSH channel-security proof.

Practical implementation and controlled migration evaluation. We use `liboqs` [23] to implement KEMUAuth as a new OpenSSH user-authentication method that coexists with signature credentials and supports standalone KEM, hybrid, and SSH-native multi-step migration configurations. We evaluate it under controlled network emulation across representative RTTs, TCP initial-window settings, and practical post-quantum authentication choices, measuring handshake latency,

transmission effects, and server-side online authentication cost. The results show that KEMUAuth closely matches Ed25519 and ML-DSA in end-to-end latency under common network settings, with clearer benefits emerging when post-quantum signatures impose larger transmission or verification costs. In particular, under small TCP initial windows, it reduces median handshake latency by 7.0–7.3% relative to the ML-DSA baseline and lowers the measured server-side online cryptographic cost by 59.1% compared with ML-DSA in the same NIST security category.

B. Related Work

Post-quantum SSH migration has produced a clear division between key exchange and authentication. For key exchange, KEM-based mechanisms have moved from prototypes into deployment and standardization. Mainline OpenSSH treats post-quantum key agreement as a baseline capability, and large-scale services such as GitHub have enabled hybrid post-quantum SSH key exchange for production access [24], [25]. Standardization has followed this direction, with RFC 9941 documenting the widely deployed `sntrup761x25519-sha512` hybrid KEX method and ongoing Internet-Drafts specifying ML-KEM-based SSH key-exchange variants [6], [7], [26]. For authentication, existing proposals mainly preserve SSH’s signature-based public-key model by adding post-quantum signature formats, including ML-DSA public keys, composite ML-DSA signatures, and SSH profiles that mandate ML-DSA-based authentication [27]–[29]. Research prototypes and measurements mirror this division, evaluating post-quantum KEX and signature-based authentication in OQS-OpenSSH and `liboqs`-based prototypes, and quantifying their latency, bandwidth, and deployment impact [10], [30]–[32].

Formal analyses of post-quantum SSH have primarily treated the transport layer as the security-critical boundary. Tran et al. give a symbolic AKE-style analysis of hybrid SSH key exchange [33], while Benčina et al. provide a post-quantum ACCE analysis of SSH that captures *harvest-now, decrypt-later* attacks [3]. Other formal verification work similarly focuses on hybrid key exchange and the confidentiality or integrity of the resulting channel [34]. These analyses provide the foundation for reasoning about post-quantum SSH transport security, but their authentication guarantees are centered on the transport-layer exchange and server authentication, rather than on proof of possession in the user-authentication layer. A related but orthogonal line is the privacy-preserving SSH authentication scheme of Roy et al., which uses anonymous multi-KEM and private set intersection to hide authorized-key membership [35]. Its goal is authorized-key privacy, rather than a general post-quantum KEM credential for standard SSH user authentication.

KEM-based authentication has been explored beyond SSH in AKE and secure-channel protocols. In TLS-style designs, KEMTLS [18] and AuthKEM [19] replace handshake signatures with certified long-term KEM public keys, while post-quantum WireGuard [17] redesigns its handshake to sup-

port KEM-based authentication. Similar directions are being considered for IKEv2/IPsec and EDHOC, including KEM-based, signature-free, and mixed signature/KEM authentication mechanisms for constrained or migration-oriented settings [36]–[38]. At the AKE-construction level, protocols such as PQuAKE [39], Muckle# [40], and HetAKE [41] study implicit, hybrid, or heterogeneous authentication mechanisms for post-quantum migration. Together, these works establish KEM-based authentication as a viable design paradigm, while also illustrating that existing approaches typically require redesigning the handshake or key schedule to place KEM credentials inside the core protocol flow.

Overall, existing post-quantum SSH work has treated KEMs primarily as transport-layer key-exchange mechanisms, while existing KEM-authentication work has largely placed KEM credentials inside AKE handshakes or key schedules. This leaves SSH’s user-authentication layer as a distinct and comparatively underexplored point in the design space for post-quantum KEM-based authentication.

II. PRELIMINARIES

A. Notation

The security parameter is denoted by $\lambda \in \mathbb{N}$. For a finite set S , $x \xleftarrow{\$} S$ denotes sampling x uniformly at random from S , and $y \leftarrow \text{Alg}(x)$ denotes assignment of the output of algorithm Alg on input x . Unless stated otherwise, probabilities are taken over the randomness of the experiment, the algorithms, and the adversary. For bit strings x and y , $x \parallel y$ denotes concatenation, and $\{0, 1\}^\ell$ denotes the set of ℓ -bit strings. The symbol $\perp$ denotes failure or rejection. PPT and QPT stand for probabilistic polynomial-time and quantum polynomial-time, respectively. We write $\text{negl}(\lambda)$ for an unspecified negligible function, and $\text{Adv}_{\Pi}^{\text{notion}}(\mathcal{A})$ for the advantage of adversary $\mathcal{A}$ against primitive or protocol Π in security notion.

B. KEMs

A key encapsulation mechanism (KEM) is a public-key primitive that allows a party to establish a shared secret with the holder of a corresponding secret key. Formally, a KEM consists of three probabilistic polynomial-time algorithms

$$\text{KEM} = (\text{KeyGen}, \text{Encap}, \text{Decap})$$

defined as follows:

- $\text{KeyGen}(1^\lambda)$ outputs a public/secret key pair (pk, sk) .
- $\text{Encap}(pk)$ outputs a ciphertext ct and a shared secret K .
- $\text{Decap}(sk, ct)$ outputs either a shared secret K' or the failure symbol $\perp$.

Correctness requires that, except with negligible probability, if $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$ and $(K, ct) \leftarrow \text{Encap}(pk)$, then $\text{Decap}(sk, ct) = K$.

The standard security requirement is indistinguishability of the encapsulated key from random. For $\text{atk} \in \{\text{cpa}, \text{cca}\}$, let $G_{\text{KEM}, \mathcal{A}}^{\text{ind-atk}}$ denote the usual KEM indistinguishability experiment for adversary $\mathcal{A}$, where $\mathcal{A}$ receives a challenge ciphertext ct^* together with either the real encapsulated key or an

independent random key. In the IND-CPA experiment, $\mathcal{A}$ has no decapsulation access, while in the IND-CCA experiment it may query a decapsulation oracle except on ct^* . The advantage of $\mathcal{A}$ is

$$\text{Adv}_{\text{KEM}}^{\text{ind-atk}}(\mathcal{A}) = \left| \Pr[G_{\text{KEM}, \mathcal{A}}^{\text{ind-atk}}(1^\lambda) = 1] - \frac{1}{2} \right|.$$

A KEM is ind-atk secure if this advantage is negligible for every efficient adversary $\mathcal{A}$.

In our SSH analysis, the ephemeral KEM used in transport-layer key exchange requires IND-CPA security, while the long-term client KEM used for authentication requires the stronger IND-CCA security. Formal experiments for these notions, together with the other cryptographic assumptions used in the analysis, are collected in Appendix A.

C. Baseline SSH Publickey Flow

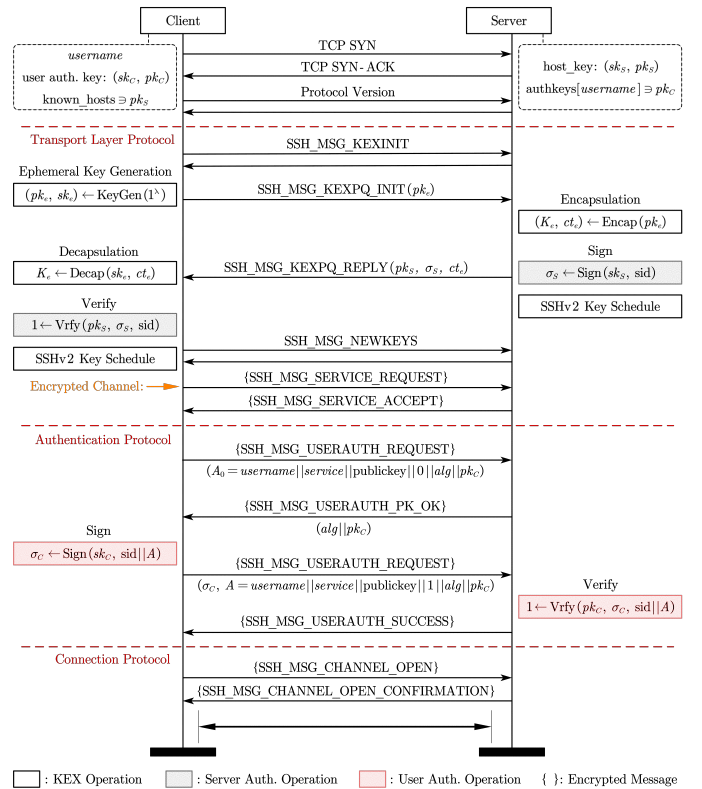


Fig. 1. Baseline SSH flow with post-quantum transport-layer key exchange and signature-based publickey user authentication. The flow follows RFC 4253 [1] and RFC 4252 [21], abstracting recent OpenSSH post-quantum key-exchange suites [24]. The key schedule is simplified: the first key exchange defines the session identifier sid as the exchange hash, which binds the version strings, KEXINIT messages, server host key, key-exchange values, and shared secret. The figure expands only the publickey authentication layer, where the client signs sid concatenated with the authentication request context A , excluding the signature field.

Figure 1 summarizes the SSH baseline considered in this work. SSH consists of the Transport Layer Protocol, the User Authentication Protocol, and the Connection Protocol. In post-quantum migration, the key-exchange and authentication primitives may be replaced by post-quantum or hybrid alternatives,

while this layering remains unchanged. The transport layer establishes the session identifier `sid` and the encrypted channel. User authentication verifies the client inside that channel. Our design targets the signature proof in the `publickey` method, the dominant baseline for public-key user authentication.

During the transport phase, the client and server exchange protocol versions, negotiate algorithms via `SSH_MSG_KEXINIT`, and perform key exchange with server host-key authentication. After `SSH_MSG_NEWKEYS`, all subsequent service-request, user-authentication, and connection-layer messages are protected by the established encrypted channel. In the standard `publickey` method, the client typically first sends an unsigned `SSH_MSG_USERAUTH_REQUEST` containing the username, requested service, method name, public-key algorithm, and public key. If the server considers the key acceptable, it replies with `SSH_MSG_USERAUTH_PK_OK`. The client then sends a signed `SSH_MSG_USERAUTH_REQUEST`, where the signature σ_C is computed over `sid` and the authentication request context A . SSH also permits a direct signed request without the preliminary acceptability query, but the two-step form is commonly used to avoid unnecessary signing or key-unlocking operations. Once user authentication succeeds, the connection protocol carries channel-opening and application data over the secure channel.

D. ACCE Security Model

We analyze our protocol in the authenticated and confidential channel establishment (ACCE) framework. An ACCE protocol first establishes peer identities, a session identifier, and channel keys during the handshake, and then uses the derived keys to protect application data with stateful authenticated encryption. This abstraction matches SSH: the transport layer establishes `sid` and channel keys, user authentication completes client authentication inside the protected channel, and the connection protocol carries application data. Our model follows the SSH ACCE formulation of [3], including its session variables, adversarial oracle interface, and matching-session structure, with the full experiment and our mutual-authentication extension specified in Appendix B.

ACCE security consists of authentication and channel security. Authentication rules out malicious acceptance. A session π_i^s accepts maliciously if it accepts believing that its peer is an uncorrupted party, but there is no unique matching session at that peer with the corresponding role, ciphersuite, and session identifier. In our SSH setting, this condition is required in both directions, meaning that the client must not accept without an honest matching server session and the server must not accept without an honest matching client session. For an adversary $\mathcal{A}$, we define

$$\text{Adv}_{\text{SSH}}^{\text{acce-mu-auth}}(\mathcal{A}) = \Pr[\exists \pi_i^s : \pi_i^s \text{ accepts maliciously}].$$

Authentication security means that this probability is negligible. The post-quantum authentication definition uses the same malicious-acceptance event, but considers QPT adversaries.

Channel security captures confidentiality and integrity after acceptance. The adversary may interact with accepted sessions through encryption and decryption queries and eventually chooses a fresh accepted test session. Freshness means, informally, that the adversary has not revealed the test session key, has not revealed the key of its matching partner, and has not corrupted the relevant peer before acceptance. The test session contains a hidden bit b that determines whether the encryption oracle returns real encryptions or challenge encryptions. If $\mathcal{A}$ outputs a guess b' , we write

$$p = \Pr[b' = b \mid \pi_i^s \text{ is a fresh accepted test session}],$$

and define

$$\text{Adv}_{\text{SSH}}^{\text{acce-mu-aenc}}(\mathcal{A}) = \left| p - \frac{1}{2} \right|.$$

Channel security requires this advantage to be negligible for any efficient adversary.

For post-quantum channel security, the experiment models “harvest now, decrypt later” attacks by restricting the handshake phase to honest executions. Instead of actively driving sessions with a send oracle, a QPT adversary obtains honestly generated transcripts through an execution oracle and may later issue reveal, corrupt, encryption and decryption queries subject to the same freshness conditions. Since these executions already have honest matching partners, the experiment focuses on confidentiality and integrity of the resulting channel. We denote the corresponding advantage by

$$\text{Adv}_{\text{SSH}}^{\text{pq-acce-aenc}}(\mathcal{Q}) = \left| p_{\text{pq}} - \frac{1}{2} \right|,$$

where p_{pq} is the probability that $\mathcal{Q}$ correctly guesses the hidden channel challenge bit for a fresh accepted session.

III. KEM-BASED USER AUTHENTICATION FOR SSH

This section presents KEMUAuth, a KEM-based SSH user-authentication method that coexists with the existing signature-based `publickey` method. Within KEMUAuth, the signature proof of possession normally used in public-key user authentication is replaced by a session-bound KEM response. As shown in Figure 2, KEMUAuth is invoked only after the transport layer has established the encrypted channel and the session identifier. It therefore leaves SSH key exchange, host-key authentication, and connection-layer semantics unchanged, and confines the change to the user-authentication proof.

At the start of user authentication, the client holds a long-term KEM key pair (sk_C, pk_C) , and the server maintains authorized public keys for each username, typically via an `authorized_keys`-style database. The client begins by sending an encrypted `SSH_MSG_USERAUTH_REQUEST` using the provisional method name `publickey-kem`. This request contains the username, requested service, KEM algorithm identifier alg , and the client’s KEM public key pk_C . The server checks whether pk_C is authorized for the claimed username and whether alg is acceptable. If either check fails, the server returns `SSH_MSG_USERAUTH_FAILURE`. Otherwise, it proceeds with the KEM challenge.

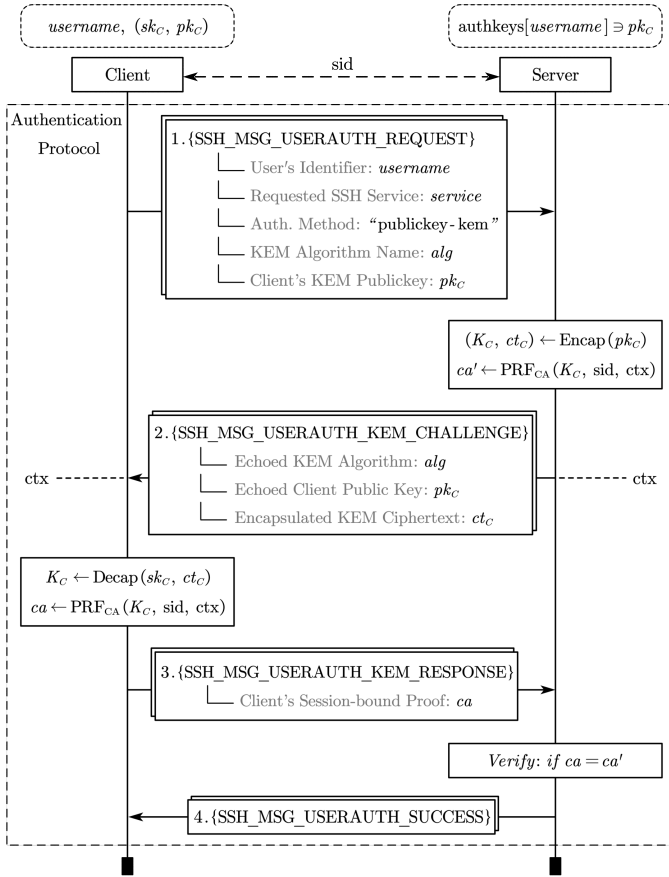


Fig. 2. KEM-based user authentication for SSH. Before this stage, the transport layer has established the session identifier sid and an encrypted channel. Messages in $\{\}$ are channel-protected. The server authorizes pk_C for the claimed username, while the client holds the corresponding secret key sk_C . Grey text denotes field annotations rather than messages.

To challenge the client, the server runs

$$(K_C, ct_C) \leftarrow \text{Encap}(pk_C)$$

and computes the expected response

$$ca' \leftarrow \text{PRF}_{CA}(K_C, sid, ctx).$$

Let msg be the KEMUAuth method transcript in Figure 2. It includes the initial request fields $(username, service, publickey-kem, alg, pk_C)$, the challenge fields (alg, pk_C, ct_C) , and the response message type, but not the response value ca . We define

$$ctx = \text{Encode}_{SSH}(msg).$$

The server sends `SSH_MSG_USERAUTH_KEM_CHALLENGE`, which echoes alg and pk_C and carries ct_C . Echoing these fields allows the client to match the challenge to the pending request encoded in ctx and avoids ambiguity across retrieved or concurrent authentication attempts.

Upon receiving the challenge, the client first checks that the echoed algorithm and public key match its pending authentication request. It then decapsulates

$$K_C \leftarrow \text{Decap}(sk_C, ct_C).$$

If decapsulation fails, the client rejects the challenge. Otherwise, it computes

$$ca \leftarrow \text{PRF}_{CA}(K_C, sid, ctx)$$

and returns ca in `SSH_MSG_USERAUTH_KEM_RESPONSE`. The server accepts if and only if $ca = ca'$, in which case it returns `SSH_MSG_USERAUTH_SUCCESS`. Otherwise, it returns `SSH_MSG_USERAUTH_FAILURE`. The explicit response is necessary because the server must verify possession of the decapsulated secret without feeding that secret into SSH transport key derivation.

We instantiate the client-authentication response function as

$$ca := \text{PRF}_{CA}(K_C, sid, ctx) = \text{HMAC}(K_C, \text{label} \parallel sid \parallel ctx),$$

where label is a fixed domain-separation label, such as "ssh-publickey-kem-client-auth". The fields in ctx are encoded using SSH's length-prefixed binary string format, so the response is computed over an unambiguous representation of the authentication attempt. The concrete response length and hash function are specified by the advertised KEMUAuth algorithm identifier.

The binding to sid and ctx gives the response the same session-binding role as the signature in the standard `publickey` method. The session identifier prevents replay across SSH sessions, while the authentication context binds the proof to the selected method, user, service, algorithm, public key, challenge ciphertext, and message flow. Since only the holder of sk_C can recover K_C from ct_C , a valid response proves possession of the authorized KEM secret key for this SSH authentication attempt. However, because the server also knows K_C and can generate the same response, the resulting transcript provides deniable online authentication rather than transferable evidence of a client login. As a result, KEMUAuth replaces the client signature with a KEM challenge-response proof, while requiring only one new authentication method and two method-specific user-authentication messages. Existing signature credentials and multi-method authentication policies remain available for incremental deployment.

IV. SECURITY ANALYSIS

We now analyze SSH with KEMUAuth, the KEM-based user-authentication method described in Section III. Since KEMUAuth leaves transport-layer key exchange, server host-key authentication, and channel-key derivation unchanged, the main new proof obligation is server acceptance of a client authenticated through the KEM-based response at the user-authentication layer. We first prove mutual ACCE authentication, then derive classical and post-quantum channel security by composing with the existing SSH ACCE analysis, and finally discuss the security requirements on PRF_{CA} .

A. ACCE Authentication Security

For the transport and server-authentication components, we adopt the assumptions of the post-quantum SSH ACCE analysis in [3]: a collision-resistant hash function H , an EUF-CMA-secure digital signature scheme DSS , a secure pseudorandom

function PRF_{SSH} , a secure stateful authenticated encryption scheme BSAE, and an IND-CPA-secure ephemeral KEM_e for transport-layer key exchange. Our client-authentication extension additionally requires the long-term client KEM_C to be IND-CCA secure and the response function PRF_{CA} to satisfy the standard PRF security notion. Formal definitions of these primitives and security notions are collected in Appendix A.

Our authentication definition follows the ACCE formulation of [3], but extends the malicious-acceptance event to mutual authentication. In [3], malicious acceptance is counted only for initiator sessions, since the analyzed mode authenticates the server to the client but not the client to the server. Since KEMUAAuth adds user-authentication-layer client authentication, we define $\text{Adv}_{\text{SSH}}^{\text{acce-mu-auth}}(\mathcal{A})$ as the probability that any initiator or responder session accepts believing that its peer is an uncorrupted party, but has no unique matching session of the opposite role.

Lemma IV.1 (SSH is Mutual ACCE-Authentication-secure). *Consider SSH with signature-based server authentication and KEMUAAuth client authentication. Let μ be the length of the random cookies/nonces exchanged during transport-layer negotiation, let n_P be the number of parties, let n_S be the maximum number of sessions per party, and let ℓ be the output length of PRF_{CA} . For any PPT adversary $\mathcal{A}$, there exist adversaries $\mathcal{B}_1, \dots, \mathcal{B}_7$ such that*

$$\begin{aligned} \text{Adv}_{\text{SSH}}^{\text{acce-mu-auth}}(\mathcal{A}) &\leq \frac{(n_P n_S)^2}{2^\mu} + \text{Adv}_{\text{H}}^{\text{coll}}(\mathcal{B}_1) \\ &\quad + n_P^2 n_S \left(\text{Adv}_{\text{DSS}}^{\text{euf-cma}}(\mathcal{B}_2) + \text{Adv}_{\text{KEM}_e}^{\text{ind-cpa}}(\mathcal{B}_3) \right. \\ &\quad + \text{Adv}_{\text{PRF}_{\text{SSH}}}^{\text{prf}}(\mathcal{B}_4) + \text{Adv}_{\text{BSAE}}^{\text{bsae}}(\mathcal{B}_5) \\ &\quad \left. + \text{Adv}_{\text{KEM}_C}^{\text{ind-cca}}(\mathcal{B}_6) + \text{Adv}_{\text{PRF}_{\text{CA}}}^{\text{prf}}(\mathcal{B}_7) + 2^{-\ell} \right). \end{aligned}$$

Proof. The proof proceeds by game hopping. Each hop either aborts on a rare event or reduces a successful malicious acceptance to the security of an underlying primitive. A complete game sequence is given in Appendix B.

We first exclude collisions in the random values exchanged during transport-layer negotiation, contributing at most $(n_P n_S)^2 / 2^\mu$. We also abort on collisions in the exchange hash, which reduces to the collision resistance of H. We then guess the first maliciously accepting session and, when needed, the peer identity involved in the impersonation; this accounts for the factor $n_P^2 n_S$ in the bound. The remaining analysis splits according to the role of the first accepting session.

a) *Case A: malicious client acceptance:* This case is the server-authentication branch of the argument. After excluding nonce and hash collisions, a client that accepts without a matching server session must either accept a fresh valid host-key signature, yielding an EUF-CMA forgery against DSS, or be led to accept through modified protected traffic. In the latter case, we replace the transport-layer KEM secret with a uniformly random value using the IND-CPA security of KEM_e , then replace the SSH key-schedule output with random channel keys using the PRF security of PRF_{SSH} . With independent random channel keys, any successful modification

of protected user-authentication messages gives an adversary against the BSAE security of the channel encryption scheme. Thus, malicious client acceptance is bounded by the host-key signature, transport KEM, SSH key schedule, and channel-encryption terms.

b) *Case B: malicious server acceptance:* This is the new authentication case introduced by our KEM-based user-authentication method. Here, a server accepts a client without a unique matching client session. The server's acceptance condition is a valid response

$$ca = \text{PRF}_{\text{CA}}(K_C, \text{sid}, \text{ctx}),$$

where $(K_C, \text{ct}_C) \leftarrow \text{Encap}(pk_C)$ is generated by the server under the authorized long-term client KEM public key pk_C .

We guess the impersonated client identity and embed an IND-CCA challenge for KEM_C into that client's long-term authentication key. The reduction uses its decapsulation oracle to simulate honest client decapsulations, except for the target challenge ciphertext used in the malicious-acceptance attempt. If the adversary can distinguish whether the target encapsulated key is real or random, this gives an IND-CCA adversary against KEM_C .

After the target encapsulated key is hidden, we replace the response derived from PRF_{CA} with a uniformly random ℓ -bit string, losing only the PRF advantage of PRF_{CA} . This replacement is applied at a fresh target input $y^* = \text{sid}^* \parallel \text{ctx}^*$, whose collisions with non-target evaluations are already covered by the nonce and exchange-hash collision bounds. The expected response is therefore independent of the adversary's view before it sends ca . Therefore, the only remaining way to make the server accept is to guess this value, which succeeds with probability at most $2^{-\ell}$. Hence malicious server acceptance is bounded by the IND-CCA security of KEM_C , the PRF security of PRF_{CA} , and the direct-guessing term.

Combining the two cases with the initial collision and guessing losses gives the stated bound. $\square$

We now consider authentication security against quantum adversaries. The malicious-acceptance event is unchanged: an initiator or responder session accepts believing that its peer is an uncorrupted party, but no unique matching session exists. Thus, for authentication security, the adversary class changes from PPT to QPT, while the interaction with the ACCE authentication experiment remains unchanged.

Theorem IV.1 (Post-quantum ACCE authentication security). *Consider SSH with signature-based server authentication and KEMUAAuth client authentication. Suppose that all primitive assumptions in Lemma IV.1 hold against quantum polynomial-time adversaries: collision resistance of H, EUF-CMA security of DSS, IND-CPA security of KEM_e , PRF security of PRF_{SSH} , authenticated-encryption security of BSAE, IND-CCA security of KEM_C , and PRF security of PRF_{CA} . Then, for every QPT*

adversary $\mathcal{Q}$, there exist adversaries $\mathcal{B}_8, \dots, \mathcal{B}_{14}$ such that

$$\begin{aligned} \text{Adv}_{\text{SSH}}^{\text{pq-acce-mu-auth}}(\mathcal{Q}) &\leq \frac{(n_P n_S)^2}{2^\mu} + \text{Adv}_{\text{H}}^{\text{coll}}(\mathcal{B}_8) \\ &\quad + n_P^2 n_S \left(\text{Adv}_{\text{DSS}}^{\text{euf-cma}}(\mathcal{B}_9) + \text{Adv}_{\text{KEM}_e}^{\text{ind-cpa}}(\mathcal{B}_{10}) \right. \\ &\quad + \text{Adv}_{\text{PRF}_{\text{SSH}}}^{\text{prf}}(\mathcal{B}_{11}) + \text{Adv}_{\text{BSAE}}^{\text{bsae}}(\mathcal{B}_{12}) \\ &\quad \left. + \text{Adv}_{\text{KEM}_C}^{\text{ind-cca}}(\mathcal{B}_{13}) + \text{Adv}_{\text{PRF}_C}^{\text{prf}}(\mathcal{B}_{14}) + 2^{-\ell} \right). \end{aligned}$$

Proof. The proof follows the game-hopping argument of Lemma IV.1, with all primitive advantages interpreted against QPT adversaries. The reductions are straight-line and black-box, since they only simulate the ACCE authentication experiment, embed the relevant primitive challenge, and forward the adversary’s output, without using random-oracle programmability, rewinding, the forking lemma, or measurements of the adversary’s internal state. By the lifting argument of Song [42], the same sequence of hops applies in the quantum setting. Malicious client acceptance reduces to breaking the host-key signature, KEM_e , PRF_{SSH} , or channel encryption scheme. Malicious server acceptance reduces to breaking the KEM_C or PRF_C , except with probability $2^{-\ell}$ for directly guessing the response. These quantities are negligible by assumption, which proves the claim. $\square$

B. ACCE Channel Security

We now derive channel security for SSH with KEM-based user authentication. The proof follows the post-quantum SSH channel-security analysis of [3].¹ This reuse is justified because our modification is confined to user authentication: all KEM-authentication messages are sent after `SSH_MSG_NEWKEYS`, inside the encrypted channel, and are not inputs to transport key exchange, session-identifier derivation, or channel-key derivation. Thus, the classical ACCE bound only replaces server-only authentication with the mutual-authentication term from Lemma IV.1.

Lemma IV.2 (Channel security, KEM-based user auth. mode). *Let μ be the length of the random cookies/nonces exchanged during transport-layer negotiation, let n_P be the number of parties, and let n_S be the maximum number of sessions per party. For any PPT adversary $\mathcal{A}$, there exist adversaries $\mathcal{B}_{15}, \mathcal{B}_{16}, \mathcal{B}_{17}$ such that*

$$\begin{aligned} \text{Adv}_{\text{SSH}}^{\text{acce-mu-aenc}}(\mathcal{A}) &\leq \text{Adv}_{\text{SSH}}^{\text{acce-mu-auth}}(\mathcal{A}) \\ &\quad + n_P^2 n_S^2 \left(\text{Adv}_{\text{KEM}_e}^{\text{ind-cpa}}(\mathcal{B}_{15}) \right. \\ &\quad \left. + \text{Adv}_{\text{PRF}_{\text{SSH}}}^{\text{prf}}(\mathcal{B}_{16}) + \text{Adv}_{\text{BSAE}}^{\text{bsae}}(\mathcal{B}_{17}) \right). \end{aligned}$$

Proof. The proof aborts if the test session accepts maliciously, an event bounded by $\text{Adv}_{\text{SSH}}^{\text{acce-mu-auth}}$. Conditioned on no such event, the test session has a unique matching partner. The proof then guesses the test session and partner, replaces the transport KEM secret using IND-CPA security of KEM_e , replaces the key-schedule output of PRF_{SSH} , and embeds the channel into a BSAE challenger, yielding the stated bound. $\square$

¹We include this statement to complete the ACCE treatment of our SSH variant and show how the channel-security bound composes with Lemma IV.1.

For post-quantum channel security, the KEM-authentication layer does not affect the proof. In the “*harvest now, decrypt later*” experiment, sessions obtained through $\text{OExecute}(i, s, j, t)$ are honestly generated and already have matching partners. The argument therefore depends only on the transport-layer KEM, the SSH key schedule, and the channel encryption scheme.

Theorem IV.2 (Post-quantum channel security). *Let n_P be the number of parties and n_S the maximum number of sessions per party. For any QPT adversary $\mathcal{Q}$, there exist QPT adversaries $\mathcal{B}_{18}, \mathcal{B}_{19}, \mathcal{B}_{20}$ such that*

$$\begin{aligned} \text{Adv}_{\text{SSH}}^{\text{pq-acce-aenc}}(\mathcal{Q}) &\leq n_P n_S \left(\text{Adv}_{\text{KEM}_e}^{\text{ind-cpa}}(\mathcal{B}_{18}) \right. \\ &\quad \left. + \text{Adv}_{\text{PRF}_{\text{SSH}}}^{\text{prf}}(\mathcal{B}_{19}) + \text{Adv}_{\text{BSAE}}^{\text{bsae}}(\mathcal{B}_{20}) \right). \end{aligned}$$

Proof. Since execution-oracle sessions are honest, no malicious-acceptance abort is needed. The proof guesses the test session, replaces the K_e and SSH key-schedule output using the quantum security of KEM_e and PRF_{SSH} , and embeds the resulting channel into a quantum-secure BSAE challenger. KEM-authentication messages occur only after channel keys are established and do not affect these game hops. $\square$

C. Security Role of PRF_C

The role of PRF_C is to turn the KEM shared secret K_C into an explicit authentication value that is bound to the SSH session and the concrete user-authentication attempt. In the malicious server-acceptance case of Lemma IV.1, the IND-CCA security of the client KEM first hides the challenge secret K_C from the adversary. The PRF security of PRF_C then allows the expected response on (sid, ctx) to be replaced by a uniformly random ℓ -bit string, leaving only the direct guessing probability $2^{-\ell}$. For post-quantum authentication security, the same requirement is interpreted against QPT distinguishers. Our HMAC-based instantiation follows Section III and relies on standard PRF assumptions for HMAC [43]–[45].

V. IMPLEMENTATION AND EVALUATION

This section evaluates KEMUAuth as a practical drop-in user-authentication method for SSH. We study whether replacing client signatures with a KEM-based proof changes the cost profile of post-quantum SSH authentication in realistic deployment settings. Our evaluation covers an OpenSSH-based implementation, primitive-level authentication costs, end-to-end handshake latency under representative RTTs, TCP initial-window effects, and server-side online authentication cost. The results show that KEMUAuth is competitive with signature-based public-key authentication in common network settings, while providing clearer benefits when large post-quantum signatures stress transmission or server-side verification. The raw experimental data are reported in Appendix D.

A. Instantiation and Implementation

We implemented KEMUAuth in an OQS-v10 fork based on OpenSSH 10.2p1 as a local extension to the SSH user-authentication layer. The prototype adds a new authentication

method, `publickey-kem`, for KEM public-key credentials, allowing it to coexist with existing SSH authentication methods. It uses `liboqs` [23] for post-quantum KEM operations and currently supports ML-KEM-512, ML-KEM-768, and ML-KEM-1024 as client credential algorithms. Following the transcript binding in Section III, the client-authentication response is instantiated as an HMAC-SHA-2 value keyed by the decapsulated KEM secret and computed over the fixed label `ssh-publickey-kem-client-auth`, the SSH session identifier, and the canonical SSH encoding of the KEMAuth authentication context. KEM choices and response-function variants are exposed through KEMAuth algorithm identifiers *alg* for future extensibility. The code, scripts, and patches used in our evaluation are publicly available.²

B. Experimental Setup

Testbed. All experiments were conducted on a workstation-class test host equipped with an Intel Core Ultra 9 275HX at 2.7GHz, configured to use 16 hardware threads with one thread per core, and 32GB RAM, running Ubuntu 22.04.5 LTS. To reduce measurement noise, we disabled CPU frequency scaling and fixed the CPU governor to `performance`. The code was compiled with GCC 11.4 using `-O2 -mavx2`. Post-quantum primitives were provided by `liboqs` 0.15.0, including AVX2-optimized implementations of ML-KEM, ML-DSA, Falcon, and SLH-DSA. Classical cryptographic algorithms, including Ed25519, were provided by OpenSSL 3.0.2.

Network configuration. Unless otherwise specified, the SSH transport-layer key exchange was fixed to `mlkem768x25519-sha256`, and the TCP maximum segment size (MSS) was set to 1460 bytes. We used RTTs of about 37 ms, 67 ms, and 163 ms as representative low-, intermediate-, and high-latency settings for the main evaluation [10]. For sensitivity analysis, we swept RTT from 0 to 200 ms and evaluated random packet-loss rates from 0% to 2%. To study TCP transmission effects, we varied the TCP initial congestion window from 3 to 50 MSS, spanning constrained small-window settings, the common IW10 setting, and larger windows observed in deployed systems [46]–[48]. All other experiments used 10 MSS. Network delay and TCP parameters were configured using Linux `tc/netem` and `ip route`.

Measurement methodology. We use two modes. For cryptographic microbenchmarks, each operation was run for 1,000 warm-up and 10,000 measured iterations, with five repetitions per experiment. We report median operation time measured using `clock_gettime(CLOCK_MONOTONIC)`. For end-to-end tests, we measure from the TCP SYN to successful user authentication over 5,000 handshakes per configuration. After each handshake, we close the connection and clear client-side cached state, including `known_hosts`, to avoid repeated-connection artifacts. We report latency in milliseconds, emphasizing medians and tail latency in the main text. Raw distributions are reported in Appendix D.

²Our implementation is available at <https://github.com/bhivfsadh/KEM-Based-User-Authentication-for-Post-Quantum-SSH>

C. Microbenchmark Characterization

Table I summarizes the primitive measurements used to interpret the protocol-level results. We select standardized or SSH-relevant baselines, with Ed25519 as the classical reference, and group post-quantum parameter sets by NIST category only to coarsely align target security strength, not to imply equivalent security. We report public-key and signature or ciphertext sizes together with signing and verification or encapsulation and decapsulation times, omitting offline key generation for long-term SSH credentials. For Falcon, whose signatures are variable-length, we report the maximum signature size. Object sizes help explain packetization effects, while local operation costs inform the server-side analysis. However, because the post-quantum schemes use AVX2-optimized `liboqs` implementations whereas Ed25519 uses OpenSSL, these measurements should not be interpreted as a general ranking of primitive families.

D. End-to-End Latency Evaluation

We next evaluate how KEMAuth affects end-to-end SSH handshake latency, with the transport key exchange fixed to `mlkem768x25519-sha256`. We first isolate the client-authentication algorithm, then study practical migration configurations across RTT regimes.

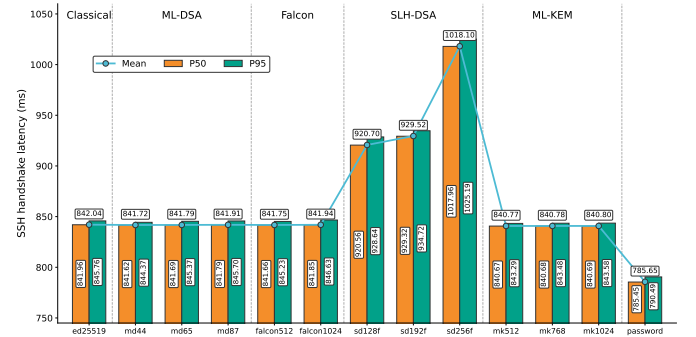


Fig. 3. End-to-end SSH handshake latency when varying only the client-authentication algorithm at RTT ≈ 67 ms. Bars show the median and 95th percentile, and the line shows the mean.

Figure 3 isolates the effect of client authentication, with Ed25519 server authentication, `mlkem768x25519-sha256` transport key exchange, and RTT fixed at approximately 67 ms. ML-KEM remains in the same latency band as compact signatures: ML-KEM-768 completes at 840.68 ms median latency, while Ed25519, ML-DSA, and Falcon remain within about 2 ms, indicating that fixed protocol and network costs dominate primitive-level differences. Password authentication is faster at 785.45 ms because it avoids the public-key probe or KEM challenge, but follows a distinct credential model rather than public-key proof of possession. In contrast, SLH-DSA-SHA2-192f and SLH-DSA-SHA2-256f reach 929.32 ms and 1017.96 ms, respectively, with the latter reflecting higher signing cost and an extra TCP flight under the default 10 MSS setting.

TABLE I
SIZES AND OPERATION COSTS OF AUTHENTICATION PRIMITIVES USED IN OUR EVALUATION

Authentication Algorithm	Notation	Problem Family	NIST PQ Category	Public Key (Bytes)	Signature/Ciphertext (Bytes)	Sign/Encaps. (ms)	Verify/Decaps. (ms)	Optimization
Ed25519 [8]	ed25519	ECC Discrete Logarithm	≈ 0 bits	32	64	0.019	0.047	OpenSSL
ML-DSA-44 [49]	md44	Module-LWE/MSIS	Level 2	1312	2420	0.033	0.014	AVX2
ML-DSA-65 [49]	md65	Module-LWE/MSIS	Level 3	1952	3309	0.054	0.022	AVX2
ML-DSA-87 [49]	md87	Module-LWE/MSIS	Level 5	2592	4627	0.066	0.035	AVX2
Falcon-512 [50]	falcon512	NTRU-SIS	Level 1	897	752	0.131	0.021	AVX2
Falcon-1024 [50]	falcon1024	NTRU-SIS	Level 5	1793	1462	0.255	0.042	AVX2
SLH-DSA-SHA2-128f [51]	sd128f	Hash-based	Level 1	32	17088	12.704	0.734	AVX2
SLH-DSA-SHA2-192f [51]	sd192f	Hash-based	Level 3	48	35664	21.380	1.122	AVX2
SLH-DSA-SHA2-256f [51]	sd256f	Hash-based	Level 5	64	49856	43.370	1.124	AVX2
ML-KEM-512 [52]	mk512	Module-LWE	Level 1	800	768	0.005	0.006	AVX2
ML-KEM-768 [52]	mk768	Module-LWE	Level 3	1184	1088	0.008	0.009	AVX2
ML-KEM-1024 [52]	mk1024	Module-LWE	Level 5	1568	1568	0.010	0.012	AVX2

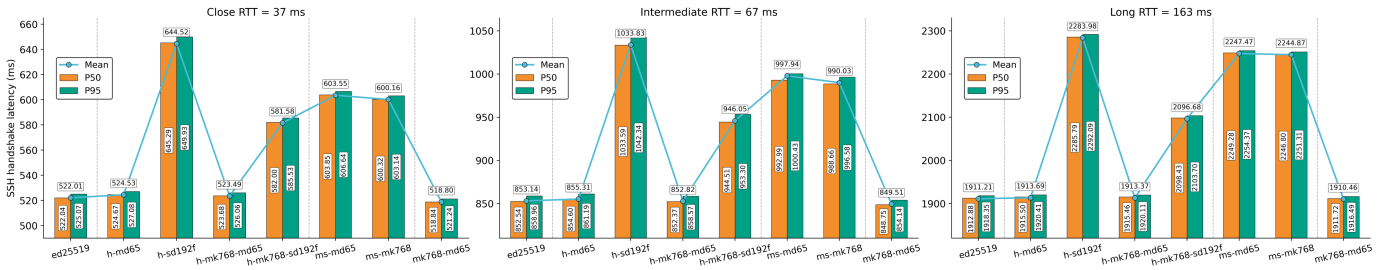


Fig. 4. End-to-end SSH handshake latency under different RTT regimes. Bars show the median and 95th percentile, and the line shows the mean. Labels without a separator indicate that the client and server use the same authentication primitive. The prefix *h*- denotes a hybrid configuration that combines Ed25519 with the indicated post-quantum authenticator. The prefix *ms*- denotes SSH-native multi-step client authentication in which Ed25519 is followed by the indicated post-quantum method, with server authentication fixed to *md65* + *ed25519*. For labels of the form *X*-*Y*, *X* denotes the client-authentication method and *Y* denotes the server-authentication method.

Figure 4 evaluates practical migration configurations across close, intermediate, and long RTT regimes, with the TCP initial window fixed at 10 MSS. These include hybrid configurations combining a classical and a post-quantum authenticator, pure post-quantum modes, and SSH-native multi-step configurations running two user-authentication methods sequentially. As expected, absolute handshake latency is largely driven by RTT, so the lower primitive cost of ML-KEM relative to ML-DSA is mostly absorbed by fixed network and protocol costs. In this setting, KEMAuth closely matches ML-DSA-based migration paths: *h*-*mk768*-*md65* is up to about 3 ms faster than the ML-DSA hybrid baseline *h*-*md65* across all RTTs. The benefit is larger when client authentication uses large SLH-DSA signatures. Both *h*-*sd192f* and *h*-*mk768*-*sd192f* use the same SLH-DSA server authenticator, while the latter replaces client-side SLH-DSA with ML-KEM. This reduces median handshake latency by 9.8%, 8.6%, and 8.2% at close, intermediate, and long RTTs, respectively. Multi-step configurations are slower than parallel hybrids because they add a serial authentication stage. The RTT-sweep and packet-loss experiments confirm that these trends remain stable under broader network conditions. Detailed results are reported in Tables VI and VII.

E. Transport Effects of Authentication Object Size

We next isolate the transmission effect of authentication-object size. The RTT is fixed at approximately 67 ms, and the TCP initial congestion window is varied from 3 to 50 MSS. This setting stresses whether public keys, signatures, and KEM ciphertexts fit into the early TCP flight, which is especially relevant for constrained or conservative TCP configurations.

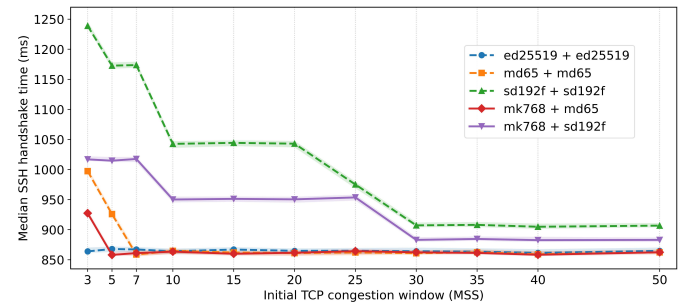


Fig. 5. SSH handshake latency under varying TCP initial congestion windows at $\text{RTT} \approx 67$ ms, with the transport key exchange fixed to *mlkem768* × *ed25519* - *sha256*. Lines show median latency and shaded bands show the 5th-95th percentile range. *X* + *Y* denotes client authentication with *X* and server authentication with *Y*.

Figure 5 shows that compact authentication configurations are relatively stable across window sizes, while configurations with large post-quantum signatures become sensitive to small initial windows. The Ed25519 baseline and `mk768 + md65` remain in the same latency band, whereas `md65 + md65` and especially `sd192f + sd192f` incur clear penalties at 3 and 5 MSS. Replacing the client signature with ML-KEM reduces this sensitivity. For ML-DSA, `mk768 + md65` lowers median latency by 7.0% at 3 MSS and 7.3% at 5 MSS compared with `md65 + md65`. At 7 MSS and above, the difference remains within 0.3% with no consistent direction. For SLH-DSA, `mk768 + sd192f` reduces median latency by 17.9% at 3 MSS and 13.5% at 5 MSS compared with `sd192f + sd192f`, and the reduction remains 8.9% at the default 10 MSS. Overall, KEMUAuth mainly benefits ML-DSA configurations under small initial windows, while the larger SLH-DSA signatures yield latency savings over a wider range of window sizes.

F. Server-Side Online Cryptographic Cost

We finally evaluate the server-side online cryptographic cost of client authentication and quantify how the resulting savings affect connection throughput under concurrent load. The measurements connect per-credential cryptographic processing with end-to-end server behavior as concurrency increases.

TABLE II
PER-CREDENTIAL ONLINE CRYPTOGRAPHIC COST

Client Auth. Notation	NIST PQ Category	Server-side Operation	Median Cost (ms)	Relative to <code>mk768</code>
<code>ed25519</code>	≈ 0 bits	Verify	0.047	$5.2\times$
<code>md44</code>	Level 2	Verify	0.014	$1.6\times$
<code>md65</code>	Level 3	Verify	0.022	$2.4\times$
<code>md87</code>	Level 5	Verify	0.035	$3.9\times$
<code>sd128f</code>	Level 1	Verify	0.734	$81.6\times$
<code>sd192f</code>	Level 3	Verify	1.122	$124.7\times$
<code>sd256f</code>	Level 5	Verify	1.124	$124.9\times$
<code>mk512</code>	Level 1	Encaps. + HMAC	0.007	$0.8\times$
<code>mk768</code>	Level 3	Encaps. + HMAC	0.009	$1.0\times$
<code>mk1024</code>	Level 5	Encaps. + HMAC	0.012	$1.3\times$

Table II shows that KEMUAuth has lower per-credential server-side cryptographic cost than all evaluated signature-verification baselines. Within NIST category 3, ML-KEM-768 plus HMAC costs 0.009 ms, 59.1% below ML-DSA-65 verification at 0.022 ms. The larger gaps against the SLH-DSA variants mainly reflect the high verification cost of hash-based signatures rather than a KEMUAuth-specific advantage.

With transport key exchange and server authentication fixed, we compared KEMUAuth with ML-DSA-65 in a concurrent-load experiment using $N \in \{1, 8, 16, 32, 64\}$ client workers. KEMUAuth improves the connection-completion rate by 4.5%, 3.3%, 1.4%, 1.1%, and 0.35%, respectively. These gains reflect its lower authentication-side cryptographic cost and narrow near CPU saturation, where common SSH costs such as key exchange, packet processing, process management,

and scheduling increasingly govern the overall connection rate. Appendix C reports the detailed experimental setup and complete throughput, latency, and memory results.

VI. DISCUSSION AND FUTURE WORK

Client-side KEM Auth. vs. Mutual KEM Auth. KEMUAuth applies KEM authentication at the user-authentication layer because SSH server authentication is embedded in the transport layer and coupled with key exchange, session binding, and host-key trust. Extending KEM authentication to the server would require broader changes and could add messages, increasing deployment complexity and eroding its benefits, especially on high-latency links. KEMUAuth therefore uses SSH’s method-extensible client-authentication layer as a clean integration point without redesigning transport semantics or adding protocol overhead.

Deniable Authentication and Auditability. KEMUAuth provides deniable online authentication through a session-bound response demonstrating possession of the client KEM secret key without creating transferable evidence. Since the server creates the challenge ciphertext and obtains the shared secret, it can reproduce the response and simulate a valid transcript. Third parties cannot attribute the transcript uniquely to the client, consistent with prior KEM-authenticated and deniable-authentication designs [18], [53]. KEMUAuth suits online access control, while deployments requiring non-repudiation or independently verifiable audit evidence should retain signature authentication alone or combine it with KEMUAuth through SSH’s multi-method policies [54], [55].

Implementation and Side-Channel Security. Our ACCE analysis establishes protocol-level security under the IND-CCA assumption for the client KEM, while leakage from concrete implementations lies outside this model. For example, ciphertexts chosen by a malicious server may expose the client’s long-term decapsulation interface as a potential decapsulation-oracle surface. Our public artifact includes preliminary malformed-ciphertext and timing checks that found no evident anomalies, but these do not constitute a comprehensive implementation-security evaluation. Deployment therefore additionally relies on constant-time decapsulation and broader side-channel and fault-injection assessment.

Toward Standardization and Interoperability. Turning KEMUAuth into an interoperable SSH extension requires a precise wire specification for method names, message formats, KEM key encodings, algorithm identifiers, transcript binding, and downgrade behavior, together with deployment integration for `authorized_keys`, SSH agents, and multi-method policies. Since KEMUAuth runs within the established SSH channel, downgrade handling primarily concerns policy fallback to other enabled authentication methods rather than tampering within the method. The prototype and controlled measurements establish feasibility, while broader adoption requires interoperability testing, machine-checked validation of the protocol state machine against implementations, and standardization within the post-quantum SSH ecosystem.

VII. CONCLUSION

We presented KEMUAuth, a drop-in KEM-based user-authentication method for post-quantum SSH that replaces client signatures with a session-bound KEM proof while preserving SSH's public-key credential model and authentication flow. We proved its security in the post-quantum ACCE framework and implemented it in OpenSSH with liboqs. Our evaluation shows competitive end-to-end performance across practical migration settings, with particular benefits when post-quantum signatures increase TCP transmission or server-side verification costs. Overall, KEMUAuth identifies a practical, lightweight middle ground for post-quantum SSH authentication, extending KEMs beyond transport-layer key exchange without redesigning SSH's transport semantics.

REFERENCES

- [1] T. Ylonen and C. Lonvick, "The Secure Shell (SSH) Transport Layer Protocol," RFC 4253, Jan. 2006. [Online]. Available: <https://www.rfc-editor.org/info/rfc4253>
- [2] F. Bergsma, B. Dowling, F. Kohlar, J. Schwenk, and D. Stebila, "Multi-ciphersuite security of the secure shell (ssh) protocol," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, 2014, pp. 369–381.
- [3] B. Benčina, B. Dowling, V. Maram, and K. Xagawa, "Post-quantum cryptographic analysis of ssh," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 595–613.
- [4] Microsoft Research, "Post-quantum ssh project," <https://www.microsoft.com/en-us/research/project/post-quantum-ssh/>, 2023, accessed: 2026-04-15.
- [5] OpenSSH Project, "OpenSSH Release Notes," <https://www.openssh.com/releases.html>, accessed: 2026-04-16.
- [6] P. Kampanakis, D. Stebila, and T. Hansen, "PQ/T Hybrid Key Exchange with ML-KEM in SSH," IETF Internet-Draft draft-ietf-sshm-mlkem-hybrid-kex-10, Feb. 2026, work in progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-sshm-mlkem-hybrid-kex/>
- [7] A. Harrison, A. Benhase, and P. Kampanakis, "Module-Lattice Key Exchange in SSH," IETF Internet-Draft draft-harrison-sshm-mlkem-01, Dec. 2025, work in progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-harrison-sshm-mlkem/>
- [8] B. Harris and L. Velvondron, "Ed25519 and Ed448 Public Key Algorithms for the Secure Shell (SSH) Protocol," RFC 8709, 2020. [Online]. Available: <https://www.rfc-editor.org/info/rfc8709>
- [9] D. Bider, "Use of RSA Keys with SHA-256 and SHA-512 in the Secure Shell (SSH) Protocol," RFC 8332, 2018. [Online]. Available: <https://www.rfc-editor.org/info/rfc8332>
- [10] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Assessing the overhead of post-quantum cryptography in tls 1.3 and ssh," in *Proceedings of the 16th International Conference on emerging Networking EXperiments and Technologies*, 2020, pp. 149–156.
- [11] T. Mallick, A. Kundu, and R. Kompella, "Study of post quantum status of widely used protocols," *arXiv preprint arXiv:2603.28728*, 2026.
- [12] H. Krawczyk, "SKEME: A versatile secure key exchange mechanism for internet," in *Proceedings of the 1996 Internet Society Symposium on Network and Distributed System Security*, 1996, pp. 114–127.
- [13] M. Bellare, R. Canetti, and H. Krawczyk, "A modular approach to the design and analysis of authentication and key exchange protocols," in *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, ser. STOC '98. ACM, 1998, pp. 419–428.
- [14] T. Perrin and M. Marlinspike, "The double ratchet algorithm," Signal Protocol Specification, 2016, accessed: 2026-05-15. [Online]. Available: <https://signal.org/docs/specifications/doubleratchet/>
- [15] T. Perrin, "The noise protocol framework," Protocol specification, revision 34, 2018, accessed: 2026-05-15. [Online]. Available: <https://noiseprotocol.org/noise.html>
- [16] J. A. Donenfeld, "WireGuard: Next generation kernel network tunnel," in *Proceedings of the Network and Distributed System Security Symposium*, ser. NDSS 2017. The Internet Society, 2017.
- [17] A. Hülsing, K. C. Ning, P. Schwabe, F. Weber, and P. R. Zimmermann, "Post-quantum WireGuard," in *2021 IEEE Symposium on Security and Privacy*, ser. SP 2021. IEEE, 2021, pp. 304–321.
- [18] P. Schwabe, D. Stebila, and T. Wiggers, "Post-quantum TLS without handshake signatures," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '20. ACM, 2020, pp. 1461–1480.
- [19] T. Wiggers, S. Celi, P. Schwabe, D. Stebila, and N. Sullivan, "KEM-based Authentication for TLS 1.3," Internet-Draft draft-celi-wiggers-tls-authkem-07, May 2026, work in progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-celi-wiggers-tls-authkem/>
- [20] T. Ylonen and C. Lonvick, "The Secure Shell (SSH) Protocol Architecture," RFC 4251, Jan. 2006. [Online]. Available: <https://www.rfc-editor.org/info/rfc4251>
- [21] T. Ylonen and C. Lonvick, "The Secure Shell (SSH) Authentication Protocol," RFC 4252, Jan. 2006. [Online]. Available: <https://www.rfc-editor.org/info/rfc4252>
- [22] F. Cusack and M. Forssen, "Generic Message Exchange Authentication for the Secure Shell Protocol (SSH)," RFC 4256, Jan. 2006. [Online]. Available: <https://www.rfc-editor.org/info/rfc4256>
- [23] Open Quantum Safe Project, "liboqs: A c library for quantum-safe cryptographic algorithms," <https://openquantumsafe.org/liboqs/>, accessed: 2026-04-16.
- [24] OpenSSH Project, "OpenSSH: Post-quantum cryptography," <https://www.openssh.org/pq.html>, accessed: 2026-05-06.
- [25] B. M. Carlson and T. Blau, "Post-quantum security for SSH access on GitHub," GitHub Blog, Sep. 2025, accessed: 2026-05-06. [Online]. Available: <https://github.blog/engineering/platform-security/post-quantum-security-for-ssh-access-on-github/>
- [26] M. Friedl, J. Mojzic, and S. Josefsson, "Secure Shell (SSH) Key Exchange Method Using Hybrid Streamlined NTRU Prime sntrup761 and X25519 with SHA-512: sntrup761x25519-sha512," RFC 9941, Apr. 2026. [Online]. Available: <https://www.rfc-editor.org/info/rfc9941>
- [27] S. Fluhrer and R. Appel, "SSH Support of ML-DSA," IETF Internet-Draft draft-sfluhrer-ssh-mldsa, Apr. 2026, work in progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-sfluhrer-ssh-mldsa/>
- [28] R. Petrov, "ML-DSA Public Key Algorithms for the Secure Shell (SSH) Protocol," IETF Internet-Draft draft-rpe-ssh-mldsa-03, Apr. 2026, work in progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-rpe-ssh-mldsa/>
- [29] A. Becker, M. J. Jenkins, and C. Wynn, "Commercial National Security Algorithm (CNSA) Suite 2.0 Profile for SSH," IETF Internet-Draft draft-becker-cnsa2-ssh-profile-03, May 2026, work in progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-becker-cnsa2-ssh-profile/>
- [30] Open Quantum Safe Project, "OQS-OpenSSH: A fork of openssh with quantum-safe cryptography," <https://github.com/open-quantum-safe/openssh>, accessed: 2026-04-15.
- [31] K. Crockett, C. Paquin, and D. Stebila, "Prototyping post-quantum and hybrid key exchange and authentication in TLS and SSH," in *NIST Post-Quantum Cryptography Standardization Conference*, 2019. [Online]. Available: <https://www.douglas.stebila.ca/research/papers/NIST-TPQC-CroPaqSte19/>
- [32] S. Zavackè and L. Buķauskas, "Practical assessment of the SSH services' transition to post-quantum cryptography," *Baltic Journal of Modern Computing*, vol. 13, no. 4, pp. 862–884, 2025.
- [33] D. D. Tran, K. Ogata, S. Escobar, S. Akleyek, and A. Otmani, "Formal analysis of post-quantum hybrid key exchange SSH transport layer protocol," *IEEE Access*, vol. 12, pp. 1672–1687, 2024.
- [34] B. Blanchet and C. Jacomme, "Post-quantum sound CryptoVerif and verification of hybrid TLS and SSH key-exchanges," in *Proceedings of the 37th IEEE Computer Security Foundations Symposium*, ser. CSF 2024. IEEE, 2024, pp. 543–556.
- [35] L. Roy, S. Lyakhov, Y. Jang, and M. Rosulek, "Practical privacy-preserving authentication for SSH," in *Proceedings of the 31st USENIX Security Symposium*, ser. USENIX Security 2022. USENIX Association, 2022. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/roy>
- [36] G. Wang and V. Smyslov, "KEM-based Authentication for IKEv2 with Post-quantum Security," Internet-Draft draft-wang-ipsecme-kem-auth-ikev2-03, Mar. 2026, work in progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-wang-ipsecme-kem-auth-ikev2/>
- [37] L. Pocero Fraile, C. Koulamas, A. P. Fournaris, and E. Haleplidis, "KEM-based Authentication for EDHOC," Internet-Draft draft-pocero-

- authkem-edhoc-02, Mar. 2026, work in progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-pocero-authkem-edhoc/>
- [38] L. Pocero Fraile, C. Koulamas, A. P. Fournaris, and E. Haleplidis, “KEM-based Authentication for EDHOC in Initiator-Known Responder (IKR) Scenarios,” Internet-Draft draft-pocero-authkem-ikr-edhoc-03, Apr. 2026, work in progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-pocero-authkem-ikr-edhoc/>
- [39] U. Blumenthal, B. Luo, S. O’Melia, G. Torres, and D. A. Wilson, “PQuAKE – Post-Quantum Authenticated Key Exchange,” Internet-Draft draft-uri-lake-pquake-00, Apr. 2025, expired Internet-Draft. [Online]. Available: <https://datatracker.ietf.org/doc/draft-uri-lake-pquake-00/>
- [40] C. Battarbee, C. Striecks, L. Perret, S. Ramacher, and K. Verhaeghe, “Quantum-safe hybrid key exchanges with KEM-based authentication,” Cryptology ePrint Archive, Paper 2025/1228, 2025. [Online]. Available: <https://eprint.iacr.org/2025/1228>
- [41] X. Lu, Y. Cheng, J. He, H. Xue, and Y. Liu, “Hetake: Heterogeneous authenticated key exchange for post-quantum migration,” in *ACISP 2026*, 2026, accepted for publication.
- [42] F. Song, “A note on quantum security for post-quantum cryptography,” in *Post-Quantum Cryptography – 6th International Workshop, PQCrypto 2014*, ser. Lecture Notes in Computer Science, M. Mosca, Ed., vol. 8772. Springer, 2014, pp. 246–265.
- [43] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-hashing for message authentication,” RFC 2104, 1997.
- [44] National Institute of Standards and Technology, “The keyed-hash message authentication code (HMAC),” National Institute of Standards and Technology, FIPS PUB 198-1, 2008.
- [45] M. Bellare, “New proofs for NMAC and HMAC: Security without collision-resistance,” in *Advances in Cryptology – CRYPTO 2006*, ser. Lecture Notes in Computer Science, vol. 4117. Springer, 2006, pp. 602–619.
- [46] C. Gomez, J. Crowcroft, and M. Scharf, “Tcp usage guidance in the internet of things (iot),” RFC Editor, RFC 9006, Mar. 2021. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9006>
- [47] J. Chu, N. Dukkkipati, Y. Cheng, and M. Mathis, “Increasing tcp’s initial window,” RFC Editor, RFC 6928, Apr. 2013. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6928>
- [48] J. R  th and O. Hohlfeld, “Demystifying tcp initial window configurations of content distribution networks,” in *Proceedings of the Network Traffic Measurement and Analysis Conference*, ser. TMA ’18. IEEE, 2018, pp. 1–8. [Online]. Available: <https://doi.org/10.23919/TMA.2018.8506549>
- [49] National Institute of Standards and Technology, “Module-Lattice-Based Digital Signature Standard,” National Institute of Standards and Technology, FIPS 204, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/204/final>
- [50] Open Quantum Safe Project, “Falcon,” liboqs Algorithm Documentation, [Online]. Available: <https://openquantumsafe.org/liboqs/algorithms/sig/falcon.html>
- [51] National Institute of Standards and Technology, “Stateless Hash-Based Digital Signature Standard,” National Institute of Standards and Technology, FIPS 205, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/205/final>
- [52] National Institute of Standards and Technology, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” National Institute of Standards and Technology, FIPS 203, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/203/final>
- [53] M. Di Raimondo, R. Gennaro, and H. Krawczyk, “Deniable authentication and key exchange,” in *Proceedings of the 13th ACM Conference on Computer and Communications Security*, ser. CCS ’06. ACM, 2006, pp. 400–409.
- [54] National Institute of Standards and Technology, “Digital signature,” NIST Computer Security Resource Center Glossary, accessed: 2026-05-16. [Online]. Available: https://csrc.nist.gov/glossary/term/digital_signature
- [55] H. Krawczyk, “SIGMA: The “SIGn-and-MAC” approach to authenticated Diffie-Hellman and its use in the IKE protocols,” in *Advances in Cryptology (CRYPTO 2003)*, ser. Lecture Notes in Computer Science, vol. 2729. Springer, 2003, pp. 400–425.
- [56] M. Bellare, T. Kohno, and C. Namprempe, “Authenticated encryption in SSH: Provably fixing the SSH binary packet protocol,” in *Proceedings of the 9th ACM Conference on Computer and Communications Security*, ser. CCS ’02. ACM, 2002, pp. 1–11.

APPENDIX A CRYPTOGRAPHIC DEFINITIONS

The security proof of our proposed scheme relies on the following cryptographic primitives and their standard security notions.

Definition A.1 (Hash Function and Collision Resistance). A hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ maps an arbitrary-length bit string to a fixed-length output of λ bits. Collision resistance requires that for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ the probability that $\mathcal{A}$ outputs two distinct messages $m \neq m'$ such that $H(m) = H(m')$ is negligible. This probability is denoted by

$$\text{Adv}_H^{\text{coll}}(\mathcal{A}) = \Pr[(m, m') \leftarrow \mathcal{A} : m \neq m' \wedge H(m) = H(m')].$$

Definition A.2 (Pseudorandom Function (PRF)). A pseudo-random function family $\text{PRF} : \mathcal{K} \times \mathcal{X} \rightarrow \{0, 1\}^\lambda$ maps a key $k \in \mathcal{K}$ and an input $x \in \mathcal{X}$ to a fixed-length output. PRF security is defined via the following experiment: the challenger chooses a random key $k \leftarrow \mathcal{K}$ and a random bit $b \in \{0, 1\}$. If $b = 0$, the adversary $\mathcal{A}$ is given oracle access to $\text{PRF}(k, \cdot)$; if $b = 1$, $\mathcal{A}$ is given access to a truly random function $R : \mathcal{X} \rightarrow \{0, 1\}^\lambda$. $\mathcal{A}$ may adaptively query the oracle and finally outputs a guess b' . The adversary wins if $b' = b$, and its advantage is defined as

$$\text{Adv}_{\text{PRF}}^{\text{prf}}(\mathcal{A}) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

Definition A.3 (Stateful Authenticated Encryption (BSAE)). A BSAE scheme consists of the following algorithms:

- $\text{StInit}()$ outputs an initial state st ;
- $\text{Enc}(k, m, st)$ takes a key k , a plaintext m , and the current state st , and outputs a ciphertext c together with an updated state st' ;
- $\text{Dec}(k, c, st)$ takes a key k , a ciphertext c , and the current state st , and outputs either a plaintext m or the rejection symbol $\perp$, along with an updated state st' .

Security comprises two aspects:

- *Ciphertext indistinguishability*: an adversary cannot distinguish ciphertexts corresponding to two equal-length plaintexts, even when allowed to adaptively choose plaintexts and observe ciphertexts (while taking state updates into account). This is typically denoted as IND-BSAE.
- *Ciphertext integrity*: an adversary cannot forge a new valid ciphertext, i.e., cannot cause the decryption algorithm to output a non- $\perp$ value that was not obtained from the encryption oracle. This is typically denoted as INT-BSAE.

The combined security (satisfying both properties) is denoted by $\text{Adv}_{\text{BSAE}}^{\text{bsae}}(\mathcal{A})$. For full details of the security experiment we refer to Bergsma et al. [56].

Definition A.4 (Digital Signature (EUF-CMA)). A signature scheme $\text{DSS} = (\text{Gen}, \text{Sign}, \text{Vrfy})$ consists of key generation, signing, and verification algorithms. Existential unforgeability under chosen message attacks (EUF-CMA) is defined via the

following experiment: the challenger generates a key pair $(pk, sk) \leftarrow \text{Gen}(1^\lambda)$ and gives pk to the adversary $\mathcal{A}$. The adversary may adaptively query a signing oracle $\text{Sign}(sk, \cdot)$ and obtains signatures on chosen messages. Eventually, $\mathcal{A}$ outputs a message m^* and a signature σ^* . The adversary wins if $\text{Vrfy}(pk, m^*, \sigma^*) = 1$ and m^* was never submitted to the signing oracle. The advantage is defined as

$$\text{Adv}_{\text{DSS}}^{\text{euf-cma}}(\mathcal{A}) = \Pr[\mathcal{A} \text{ outputs a valid forgery}].$$

Definition A.5 (Key Encapsulation Mechanism (KEM)). A KEM consists of three algorithms:

- $\text{KeyGen}(1^\lambda)$ outputs a public key pk and a private key sk ;
- $\text{Encap}(pk)$ outputs a shared secret K and a ciphertext ct ;
- $\text{Decap}(sk, ct)$ outputs a shared secret K or the rejection symbol $\perp$.

Correctness requires that for all $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$ and $(K, ct) \leftarrow \text{Encap}(pk)$, we have $\Pr[\text{Decap}(sk, ct) = K] \geq 1 - \delta$, where δ is negligible.

Two security notions are standard:

- **IND-CPA security:** The challenger generates (pk, sk) , computes $(K_0, ct^*) \leftarrow \text{Encap}(pk)$, and picks uniformly random K_1 . It then chooses a random bit $b \in \{0, 1\}$ and gives (pk, ct^*, K_b) to the adversary $\mathcal{A}$. $\mathcal{A}$ outputs a guess b' and wins if $b' = b$. The advantage is defined as

$$\text{Adv}_{\text{KEM}}^{\text{ind-cpa}}(\mathcal{A}) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

In this game the adversary has no access to a decapsulation oracle.

- **IND-CCA security:** The game is the same as IND-CPA, except that the adversary may additionally adaptively query a decapsulation oracle $\text{Decap}(sk, \cdot)$ (with the restriction that it cannot query ct^*). The adversary outputs a guess b' and wins if $b' = b$. The advantage is defined as

$$\text{Adv}_{\text{KEM}}^{\text{ind-cca}}(\mathcal{A}) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

APPENDIX B

REDUCTIONIST SECURITY ANALYSIS

This appendix provides a formal security model for the SSH protocol extended with KEM-based client authentication, following the ACCE (Authenticated and Confidential Channel Establishment) framework [2], [3]. The model captures both classical and post-quantum adversaries, the latter via the OExecute oracle for “harvest now, decrypt later” attacks.

A. Model Syntax

Let n_P be the number of parties and denote the set of parties by $P_1, \dots, P_{n_P}$. Each party P_i generates a long-term key pair (sk_i, pk_i) using a key generation algorithm KeyGen . In the server-only authentication part of the handshake, the server holds a signing key; for client authentication, the client holds

a static KEM key pair. All public keys are assumed to be available to the adversary.

A session is a single protocol execution at a party. The s -th session at party P_i is denoted π_i^s .

Definition B.1 (Per-session variables). Each session maintains the following variables:

- $\rho \in \{\text{init}, \text{resp}\}$: the role of the session owner.
- $\text{pid} \in \{1, \dots, n_P, \perp\}$: the identifier of the alleged peer ($\perp$ means unauthenticated).
- $\text{status} \in \{\text{active}, \text{reject}, \text{accept}\}$: the current status.
- k : the session key, either $\perp$ or a tuple of subkeys. In SSH, $k = (k_e, k_d)$ where k_e and k_d are bidirectional authenticated encryption keys.
- sid : a session identifier defined by the protocol, used to match communicating sessions.
- s_e, s_d : state for the stateful authenticated encryption and decryption algorithms.
- Additional protocol-specific state (e.g., nonces exchanged, handshake message counters).

For the security experiment, each session also initialises a random bit $\pi_i^s.b \leftarrow \{0, 1\}$ that is used in the channel security challenge.

Definition B.2 (ACCE protocol). An ACCE protocol is a tuple of algorithms:

- $\text{KeyGen}(1^\lambda) \rightarrow (sk, pk)$: outputs a long-term key pair.
- Handshake algorithms $\text{Alg}_I, \text{Alg}_R$ (for initiator and responder): on input (sk, pk) and an incoming message m , they update the session state and return an outgoing message m' . Eventually they set $\pi.\text{pid}$, $\pi.\text{status}$, $\pi.k$, $\pi.\text{sid}$.
- Stateful authenticated encryption $\text{Enc}(\pi.k_e, m, \pi.s_e) \rightarrow (C, \pi.s'_e)$.
- Stateful authenticated decryption $\text{Dec}(\pi.k_d, C, \pi.s_d) \rightarrow (m', \pi.s'_d)$.

All algorithms implicitly take global protocol parameters, including the list of all trusted peer public keys.

B. Adversary interaction

The adversary $\mathcal{A}$ (or $\mathcal{Q}$ in the post-quantum setting) controls all network communication. It can direct parties to initiate sessions, deliver messages, reorder, alter, or drop packets. The adversary interacts with honest parties via the following oracles.

- $\text{OSend}(i, s, m)$: The adversary sends message m to session π_i^s . If π_i^s does not exist, it is created with $\rho = \text{init}$ and $\text{status} = \text{active}$ (using the special symbol $m = \text{init}$). The session processes m , updates its state, and returns the outgoing protocol message (or $\perp$ if the session terminates). This oracle models active adversarial control over the network.
- $\text{OExecute}(i, s, j, t)$: The challenger executes a full honest protocol run between two parties P_i and P_j , creating sessions π_i^s and π_j^t with matching roles ($\rho = \text{init}$ for i , $\rho = \text{resp}$ for j). All handshake messages are

generated honestly; the adversary only receives the complete transcript of the session (the sequence of messages exchanged). This oracle models *passive* eavesdropping, in particular “harvest now, decrypt later” attacks. Unlike OSend, the adversary cannot modify or inject messages into the sessions created by OExecute. In the post-quantum ACCE model, OSend is replaced by OExecute; in the classical model both oracles may be available.

- OReveal(i, s): If π_i^s has accepted (i.e., $\pi_i^s.\text{status} = \text{accept}$), the oracle returns the session key $\pi_i^s.k$ to the adversary. This models leakage of session keys (e.g., via compromise of the host).
- OCorrupt(i): The adversary obtains the long-term private key sk_i of party P_i . After this query, the party is considered *corrupted*. This models long-term key compromise.
- OEncrypt(i, s, m): If session π_i^s has accepted and is in the channel phase, the oracle returns a ciphertext $c \leftarrow \text{Enc}(\pi_i^s.k_e, m, \pi_i^s.s_e)$ and updates the encryption state. This is used in the channel security experiment to provide encryption queries.
- ODecrypt(i, s, c): If session π_i^s has accepted and is in the channel phase, the oracle returns the decryption $(m', \pi_i^s.s_d) \leftarrow \text{Dec}(\pi_i^s.k_d, c, \pi_i^s.s_d)$ (or $\perp$ if the ciphertext is invalid). This provides decryption queries.

The adversary may adaptively interleave these queries. The security experiments impose freshness conditions (e.g., no OReveal on the test session or its partner, no OCorrupt before acceptance, etc.) as defined in the channel security definitions.

C. ACCE Security

We now define the security properties for the SSH protocol with mutual authentication (server authenticated via signatures, client authenticated via KEM-based challenge-response). The definitions follow the ACCE framework, separating authentication and channel security.

Definition B.3 (Matching sessions). *We say that session π_j^t matches π_i^s if:*

1. $\pi_i^s.\rho \neq \pi_j^t.\rho$ (the roles are opposite);
2. $\pi_i^s.c = \pi_j^t.c$ (the same ciphersuite is used);
3. $\pi_i^s.\text{sid}$ prefix-matches $\pi_j^t.\text{sid}$, i.e., either:
 - if π_i^s sent the last message in $\pi_i^s.\text{sid}$, then $\pi_j^t.\text{sid}$ is a prefix of $\pi_i^s.\text{sid}$; or
 - if π_j^t sent the last message in $\pi_i^s.\text{sid}$, then $\pi_i^s.\text{sid} = \pi_j^t.\text{sid}$.

In our analysis we consider the standard SSH handshake (without abbreviated handshake), so matching sessions will have identical session identifiers: $\pi_i^s.\text{sid} = \pi_j^t.\text{sid}$.

Definition B.4 ((Post-quantum) Authentication security). *Let π_i^s be a session. We say that π_i^s accepts maliciously if:*

- $\pi_i^s.\text{status} = \text{accept}$;
- $\pi_i^s.\text{pid} = j \neq \perp$ and no OCorrupt(j) query was issued before π_i^s accepted;
- but there is no unique session π_j^t that matches π_i^s .

For mutual authentication, both initiator (client) and responder (server) sessions can accept maliciously. Define $\text{Adv}_{\text{SSH}}^{\text{acce-mu-auth}}(\mathcal{A})$ as the probability that, when a PPT adversary $\mathcal{A}$ terminates in the ACCE experiment, there exists any session (initiator or responder) that has accepted maliciously.

The post-quantum mutual-authentication experiment is defined identically, except that the adversary is a quantum polynomial-time adversary $\mathcal{Q}$. We define $\text{Adv}_{\text{SSH}}^{\text{pq-acce-mu-auth}}(\mathcal{Q})$ as the probability that, when $\mathcal{Q}$ terminates in the same ACCE authentication experiment, there exists any session, initiator or responder, that has accepted maliciously.

Definition B.5 (Channel security). *Suppose a PPT adversary $\mathcal{A}$ has access to OSend, OExecute, OReveal, OCorrupt, OEncrypt, ODecrypt. At the end of the experiment, $\mathcal{A}$ outputs (i, s, b') . We say that $\mathcal{A}$ answers the encryption challenge correctly if:*

- $\pi_i^s.\text{status} = \text{accept}$;
- No OCorrupt(j) query was issued before π_i^s accepted where $\pi_i^s.\text{pid} = j$ and π_i^s does **not** have a matching session;
- No OReveal(i, s) query was issued;
- For any π_j^t that matches π_i^s , no OReveal(j, t) query was issued;
- $\pi_i^s.b = b'$.

Define $\text{Adv}_{\text{SSH}}^{\text{acce-aenc}}(\mathcal{A}) = |p - \frac{1}{2}|$, where p is the probability that $\mathcal{A}$ answers the encryption challenge correctly and either $\pi_i^s.\rho = \text{init}$ or both $\pi_i^s.\rho = \text{resp}$ and a matching session exists.

In the post-quantum setting, the adversary is a quantum polynomial-time (QPT) algorithm and the OSend oracle is replaced by OExecute. This models passive eavesdropping with future quantum decryption capability.

Definition B.6 (Post-quantum channel security). *Let $\mathcal{Q}$ be a QPT adversary with access to OExecute, OReveal, OCorrupt, OEncrypt, ODecrypt. The experiment is the same as in Definition B.5, except that $\mathcal{Q}$ cannot use OSend (all honest sessions are generated via OExecute). We say $\mathcal{Q}$ answers the encryption challenge correctly under the same conditions. Define $\text{Adv}_{\text{SSH}}^{\text{pq-acce-aenc}}(\mathcal{Q}) = |p - \frac{1}{2}|$ analogously.*

Because OExecute guarantees that every session has a matching partner (as it creates both sides honestly), authentication is automatically satisfied. Therefore post-quantum ACCE security reduces to channel security only.

D. Mutual Auth. Mode Security

Proof. We define break_k as the event that a session accepts maliciously (Definition B.4) in Game k . The proof proceeds through a sequence of games, following the protocol flow and state definitions depicted in Figure 6.

Game 0. This is the original ACCE authentication experiment where the adversary $\mathcal{A}$ interacts with the real protocol. Hence

$$\text{Adv}_{\text{SSH}}^{\text{acce-mu-auth}}(\mathcal{A}) = \Pr[\text{break}_0].$$

Game 1. The challenger aborts if any nonces (from KEXINIT or KEXREPLY) collide. There are at most n_{PNS}

Negotiation	Signed KEX (for both modes)
1. init → resp: KEXINIT 1: $r_C \leftarrow \{0, 1\}^\mu$ 2: send KEXINIT $\leftarrow (r_C, \vec{SP}_C)$ 3: $\pi.\rho \leftarrow \text{init}$ 4: $\pi.\text{status} \leftarrow \text{active}$	4. init → resp: KEX_KEM_INIT 1: $(pk, sk) \leftarrow \text{KEM}_{\pi.c}.\text{Gen}()$ 2: send KEX_KEM_INIT $\leftarrow pk$
2. resp → init: KEXREPLY 1: $r_S \leftarrow \{0, 1\}^\mu$ 2: send KEXREPLY $\leftarrow (r_S, \vec{SP}_S)$ 3: $\pi.\rho \leftarrow \text{resp}$ 4: $\pi.\text{status} \leftarrow \text{active}$ 5: $\pi.c \leftarrow \text{neg}(\vec{SP}_C, \vec{SP}_S)$	5. resp → init: KEX_KEM_REPLY and NEWKEYS 1: $(K, ct) \leftarrow \text{KEM}_{\pi.c}.\text{Encap}(pk)$ 2: $x \leftarrow V_C \parallel V_S \parallel \text{KEXINIT} \parallel \text{KEXREPLY} \parallel pk_{S,\pi.c} \parallel pk \parallel ct$ 3: $(\pi.\text{sid}, \pi.k) \leftarrow \text{PRF}_{\text{SSH}}(K, x)$ 4: $\sigma_S \leftarrow \text{DSS}_{\pi.c}.\text{Sign}(sk_{S,\pi.c}, \pi.\text{sid})$ 5: send KEX_KEM_REPLY $\leftarrow (ct, pk_{S,\pi.c}, \sigma_S)$ 6: send NEWKEYS
3. init 1: $\pi.c \leftarrow \text{neg}(\vec{SP}_C, \vec{SP}_S)$	6. init → resp: NEWKEYS 1: $K \leftarrow \text{KEM}_{\pi.c}.\text{Decap}(sk, ct)$ 2: $x \leftarrow V_C \parallel V_S \parallel \text{KEXINIT} \parallel \text{KEXREPLY} \parallel pk_{S,\pi.c} \parallel pk \parallel ct$ 3: $(\pi.\text{sid}, \pi.k) \leftarrow \text{PRF}_{\text{SSH}}(K, x)$ 4: if $\text{DSS}_{\pi.c}.\text{Vrfy}(pk_{S,\pi.c}, \sigma_S, \pi.\text{sid}) = \perp$ then 5: $\pi.\text{status} \leftarrow \text{reject and terminate}$ 6: $\pi.\text{pid} \leftarrow S$, the owner of $pk_{S,\pi.c}$ 7: send NEWKEYS

Mutual Auth. Mode	
7. init → resp: AUTH_REQUEST 1: $A \leftarrow \text{username} \parallel \text{service} \parallel \text{publickey-kem} \parallel \text{alg} \parallel pk_{C,\pi.c}$ 2: $/ \text{alg specifies } \text{KEM}'_{\pi.c}$ 3: send AUTH_REQUEST $\leftarrow A$	10. resp → init: AUTH_SUCCESS or AUTH_FAILURE 1: $\text{ctx}' \leftarrow \text{AUTH_REQUEST} \parallel A'$ 2: $y' \leftarrow \pi.\text{sid} \parallel \text{ctx}'$ 3: $ca' \leftarrow \text{PRF}_{\text{CA}}(K', y')$ 4: if $ca' \neq ca$ then 5: $\pi.\text{status} \leftarrow \text{reject}$ 6: if $\pi.\text{status} = \text{active}$ then 7: $\pi.\text{status} \leftarrow \text{accept}$ 8: if $\pi.\text{status} = \text{accept}$ then 9: send AUTH_SUCCESS 10: if $\pi.\text{status} = \text{reject}$ then 11: send AUTH_FAILURE and terminate
8. resp → init: AUTH_CHALLENGE or AUTH_FAILURE 1: if $\text{Check}(\text{username}, \text{service}, \text{publickey-kem})$ then 2: $\pi.\text{status} \leftarrow \text{reject and terminate}$ 3: if $\pi.\text{status} = \text{active}$ then 4: $(K', ct') \leftarrow \text{KEM}'_{\pi.c}.\text{Encap}(pk_{C,\pi.c})$ 5: $A' \leftarrow \text{alg} \parallel pk_{C,\pi.c} \parallel ct'$ 6: send AUTH_CHALLENGE $\leftarrow A'$ 7: if $\pi.\text{status} = \text{reject}$ then 8: send AUTH_FAILURE and terminate	11. init 1: if AUTH_FAILURE then 2: $\pi.\text{status} \leftarrow \text{reject and terminate}$ 3: if AUTH_SUCCESS then 4: $\pi.\text{status} \leftarrow \text{accept}$
9. init → resp: AUTH_RESPONSE 1: $K' \leftarrow \text{KEM}'_{\pi.c}.\text{Decap}(sk_{C,\pi.c}, ct')$ 2: $\text{ctx} \leftarrow A \parallel \text{AUTH_CHALLENGE}$ 3: $y \leftarrow \pi.\text{sid} \parallel \text{ctx}$ 4: $ca \leftarrow \text{PRF}_{\text{CA}}(K', y)$ 5: send AUTH_RESPONSE $\leftarrow ca$	

Fig. 6. Description of post-quantum SSH handshake protocol with KEM-based Mutual Auth. Mode. In steps 1 and 2, algorithm neg negotiates a ciphersuite to be used in the protocol. V_C and V_S are version strings. In steps 5 and 6, PRF_{SSH} is defined in [3]. In step 8, algorithm Check checks the accessibility of username for service via some authentication mechanism, none or publickey-kem. In step 9, PRF_{CA} is defined in Section IV-C. $\text{ctx} \leftarrow A \parallel A'$ denotes the canonical $\text{Encode}_{\text{SSH}}(\text{msg})$ defined in Section III, including the authentication request, the challenge containing ct' , and the AUTH_RESPONSE message type. We note that from steps 7–11, their protocol is run within BSAE using $\pi.k$.

sessions, each sampling a nonce of length μ . The probability of a collision is at most $(n_P n_S)^2 / 2^{\mu+1}$. Thus

$$\Pr[\text{break}_0] \leq \Pr[\text{break}_1] + \frac{(n_P n_S)^2}{2^\mu}.$$

Game 2. The challenger maintains a list of all inputs and outputs of the hash function H . If two distinct inputs produce the same output, the game aborts and we output the collision to a collision-resistance challenger. Therefore

$$\Pr[\text{break}_1] \leq \Pr[\text{break}_2] + \text{Adv}_H^{\text{coll}}(\mathcal{B}_1),$$

where $\mathcal{B}_1$ is a reduction against the collision resistance of H .

Game 3. The challenger guesses the first session π_i^s that will accept maliciously. There are n_P parties and at most n_S sessions per party, so the guess is correct with probability $1/(n_P n_S)$. Hence

$$\Pr[\text{break}_2] \leq n_P n_S \cdot \Pr[\text{break}_3].$$

From now on we fix this session π_i^s . Its role (initiator or responder) determines which case we consider. Note that $\pi_i^s.\text{pid} = j \neq \perp$ and no $\text{OCorrupt}(j)$ query was issued before π_i^s accepted.

a) Case A: π_i^s is a client session (initiator): In this case the client accepts but there is no matching server session. We focus on the event $\text{break}_3 \wedge \text{client}$, i.e., the first malicious acceptance occurs at a client session. For brevity, we denote this event by break_{3c} . The client verifies a server signature σ_S on the session identifier $\pi.\text{sid}$. Because nonce and hash collisions are excluded by Games 1 and 2, each sid is unique. We now proceed with a sequence of games internal to this case.

Game A1. We abort the simulation if the test session π_i^s accepts after receiving a signature that was never output by a session with a matching session identifier. Since nonce and hash collisions have been excluded, all values to be signed are distinct. Hence any such abort corresponds to a signature forgery.

We construct a reduction $\mathcal{B}_2$ that simulates the protocol as in Game 3. $\mathcal{B}_2$ receives a public key pk^* from an EUF-CMA signature challenger, guesses the server identity $j = \pi_i^s.\text{pid}$ (cost factor n_P), and sets P_j 's public key to pk^* . For all other servers keys are generated normally. Whenever $\mathcal{B}_2$ needs to sign a message on behalf of P_j , it queries the EUF-CMA signing oracle. If the test session π_i^s accepts maliciously, then the pair $(\pi.\text{sid}, \sigma_S)$ is a valid forgery, which $\mathcal{B}_2$ outputs to its challenger. Therefore

$$\Pr[\text{break}_{3c}] \leq \Pr[\text{break}_{A1}] + n_P \cdot \text{Adv}_{\text{DSS}}^{\text{euf-cma}}(\mathcal{B}_2).$$

From this point onward, the client's acceptance implies that the ephemeral KEM ciphertext ct is authentic because it is bound by the server's signature.

Game A2. In this game, we replace the transport-layer KEM secret K used by the target client session π_i^s and its server partner π_j^t with a uniformly random value K^* . We construct a reduction $\mathcal{B}_3$ that interacts with $\mathcal{A}$ and embeds an IND-CPA challenge into the transcript of these sessions.

$\mathcal{B}_3$ participates in the IND-CPA experiment for KEM_e and receives the challenge public key pk^* together with a challenge ciphertext ct^* and challenge key K^* . When the target client session π_i^s would generate its ephemeral KEM_e key pair, $\mathcal{B}_3$ uses pk^* as the client's public key; no corresponding secret key is required in the simulation. When the partner server session π_j^t would encapsulate to this key, $\mathcal{B}_3$ supplies (K^*, ct^*) as the encapsulation output. At the corresponding client decapsulation step, $\mathcal{B}_3$ directly supplies the same value K^* . Because malicious acceptance has already been excluded, the target sessions communicate without adversarial modification, and the challenge ciphertext received by π_i^s is exactly ct^* .

If the challenge bit b is 0, then (ct^*, K^*) is distributed as an honest encapsulation under pk^* , and the simulation is identical to Game A1. If $b = 1$, then K^* is uniformly random and independent of ct^* , and the simulation is identical to Game A2. Hence

$$\Pr[\text{break}_{A1}] \leq \Pr[\text{break}_{A2}] + \text{Adv}_{\text{KEM}_e}^{\text{ind-cpa}}(\mathcal{B}_3).$$

Game A3. We replace the values $H, k_1, \dots, k_6$ computed by π_i^s and π_j^t as $\text{PRF}_{\text{SSH}}(K^*, \text{sid})$ with independent uniformly random strings $H^*, k_1^*, \dots, k_6^*$. Let $S = H \| k_1 \| \dots \| k_6$ and $S^* = H^* \| k_1^* \| \dots \| k_6^*$. A reduction $\mathcal{B}_4$ plays the PRF security game against PRF_{SSH} : it simulates the protocol as in Game A2, but for the test session and its partner it forwards the PRF computation to its own challenger. If the challenger returns the real PRF output we are in Game A2; if it returns a random string we are in Game A3. Thus

$$\Pr[\text{break}_{A2}] \leq \Pr[\text{break}_{A3}] + \text{Adv}_{\text{PRF}_{\text{SSH}}}^{\text{prf}}(\mathcal{B}_4).$$

Game A4. Now the keys $k_1^*, \dots, k_6^*$ are uniformly random and independent of the handshake. We replace the stateful authenticated encryption used in the channel phase by a BSAE challenger. A reduction $\mathcal{B}_5$ initialises a BSAE challenger and forwards all Enc and Dec operations of the test session π_i^s and its matching partner π_j^t to this challenger. The challenger uses either real encryption/decryption (if its internal bit is 0) or returns random ciphertexts (if the bit is 1). This replacement is sound because the keys are already random.

If π_i^s accepts maliciously after this game, the adversary must have forged a valid ciphertext under the BSAE scheme, which would break BSAE security. Hence

$$\Pr[\text{break}_{A3}] \leq \Pr[\text{break}_{A4}] + \text{Adv}_{\text{BSAE}}^{\text{bsae}}(\mathcal{B}_5).$$

In Game A4, all signatures are generated by legitimate parties, all session identifiers are unique, and the channel keys are independent random strings. The client π_i^s can only accept if it receives a valid server signature (already excluded by Game A1) and if the channel messages are authentic under the BSAE scheme. The BSAE challenger guarantees that no adversary can forge a ciphertext, so break_{A4} occurs with probability 0. Therefore

$$\Pr[\text{break}_{A4}] = 0.$$

Collecting the inequalities for Case A, we obtain

$$\begin{aligned} \Pr[\text{break}_3 \wedge \text{client}] &\leq n_P \left(\text{Adv}_{\text{DSS}}^{\text{euf-cma}}(\mathcal{B}_2) \right. \\ &\quad + \text{Adv}_{\text{KEM}_e}^{\text{ind-cpa}}(\mathcal{B}_3) \\ &\quad + \text{Adv}_{\text{PRF}_{\text{SSH}}}^{\text{prf}}(\mathcal{B}_4) \\ &\quad \left. + \text{Adv}_{\text{BSAE}}^{\text{bsae}}(\mathcal{B}_5) \right). \end{aligned}$$

b) Case B: π_i^s is a server session (responder): Now the server accepts but there is no matching client session. We consider the event $\text{break}_3 \wedge \text{server}$, denoted break_{3s} . In our mutual-authentication protocol, the server verifies a client authentication value $ca = \text{PRF}_{\text{CA}}(K_C, y)$, where (K_C, ct_C) is generated by encapsulating to the client's authorized long-term KEM public key and $y = \text{sid} \parallel \text{ctx}$ is the session-bound PRF input.

Game B1. We guess the client identity $j = \pi_i^s.\text{pid}$; this is correct with probability $1/n_P$. We replace the static KEM_C public key of client P_j with a challenge public key pk^* obtained from an IND-CCA challenger. In the target authentication attempt, the server's encapsulation to pk^* is simulated using the challenge ciphertext and challenge key. For other honest sessions involving P_j , any required decapsulation of non-challenge ciphertexts is answered using the IND-CCA decapsulation oracle. If $\mathcal{A}$ distinguishes this game from the previous one, we obtain an adversary against the IND-CCA security of KEM_C . Hence

$$\Pr[\text{break}_{3s}] \leq n_P \left(\Pr[\text{break}_{B1}] + \text{Adv}_{\text{KEM}_C}^{\text{ind-cca}}(\mathcal{B}_6) \right).$$

Game B2. In this game, the server's expected authentication value $ca' = \text{PRF}_{\text{CA}}(K_C, y)$ for the target authentication attempt is replaced by a uniformly random string of the same length. Since the target key K_C is hidden from the adversary after the IND-CCA replacement, this hop reduces to the PRF security of PRF_{CA} . A reduction $\mathcal{B}_7$ simulates the game and forwards the relevant query to a PRF challenger. Thus

$$\Pr[\text{break}_{B1}] \leq \Pr[\text{break}_{B2}] + \text{Adv}_{\text{PRF}_{\text{CA}}}^{\text{prf}}(\mathcal{B}_7).$$

Game B3. Let

$$y^* = \pi^*.\text{sid} \parallel \text{ctx}^*$$

be the input to PRF_{CA} in the target authentication attempt. Here $\text{ctx}^* \leftarrow A^* \parallel A'^*$ is the common authentication context constructed for the target attempt. It abstracts the canonical encoding $\text{Encode}_{\text{SSH}}(\text{msg}^*)$ defined in Section III, including the authentication request and challenge ciphertext, while $\pi^*.\text{sid}$ is established by the corresponding transport session. Although the reduction may need to evaluate PRF_{CA} when simulating honest non-target authentication responses, we condition on the event that y^* is not evaluated before $\mathcal{A}$ sends its response ca . In the absence of a matching client session, such a query would require another simulated authentication attempt to use the same authentication context and the same session identifier. The former fixes the target challenge and method transcript, while the latter is derived from the transport-layer

exchange and contains the randomness already protected by the nonce-collision and exchange-hash-collision exclusions in the previous games. Thus, the probability that y^* is hit through a non-target simulation is negligible and is absorbed into the preceding bad-event bounds.

The value ca' is therefore a uniformly random ℓ -bit string and is independent of the adversary's view before it sends ca . Hence,

$$\Pr[\text{Guess}] \leq 2^{-\ell}.$$

Since this game only records an event and does not change the simulation,

$$\Pr[\text{break}_{B2}] = \Pr[\text{break}_{B3}].$$

Game B4. This game is identical to Game B3, except that the target server session always rejects the received authentication response. Games B3 and B4 differ only if Guess occurs. Therefore,

$$\Pr[\text{break}_{B3}] \leq \Pr[\text{break}_{B4}] + 2^{-\ell}.$$

In Game B4, the target server session cannot accept maliciously, and hence

$$\Pr[\text{break}_{B4}] = 0.$$

Combining the steps for Case B, we obtain

$$\begin{aligned} \Pr[\text{break}_3 \wedge \text{server}] &\leq n_P \left(\text{Adv}_{\text{KEM}_C}^{\text{ind-cca}}(\mathcal{B}_6) \right. \\ &\quad \left. + \text{Adv}_{\text{PRF}_{\text{CA}}}^{\text{prf}}(\mathcal{B}_7) + 2^{-\ell} \right). \end{aligned}$$

c) Result analysis: Combining the two cases with the losses from Games 1–3, we have

$$\begin{aligned} \text{Adv}_{\text{SSH}}^{\text{acce-mu-auth}}(\mathcal{A}) &\leq \frac{(n_P n_S)^2}{2^\mu} + \text{Adv}_{\text{H}}^{\text{coll}}(\mathcal{B}_1) \\ &\quad + n_P^2 n_S \left(\text{Adv}_{\text{DSS}}^{\text{euf-cma}}(\mathcal{B}_2) + \text{Adv}_{\text{KEM}_e}^{\text{ind-cpa}}(\mathcal{B}_3) \right. \\ &\quad + \text{Adv}_{\text{PRF}_{\text{SSH}}}^{\text{prf}}(\mathcal{B}_4) + \text{Adv}_{\text{BSAE}}^{\text{bsae}}(\mathcal{B}_5) \\ &\quad \left. + \text{Adv}_{\text{KEM}_C}^{\text{ind-cca}}(\mathcal{B}_6) + \text{Adv}_{\text{PRF}_{\text{CA}}}^{\text{prf}}(\mathcal{B}_7) + 2^{-\ell} \right). \end{aligned}$$

This completes the proof. $\square$

APPENDIX C

CONCURRENT SERVER EVALUATION

We compare KEMUAuth with ML-DSA-65 as the client-authentication method while fixing the transport key exchange to `mlkem768x25519-sha256` and the server host-key algorithm to `ssh-mldsa-65`. The tests run over loopback without emulated delay or packet loss. The complete `sshd` process tree is pinned to two dedicated CPU cores and placed in a `cgroup v2`, while the client workers use a disjoint CPU set. Connection reuse and non-tested authentication methods are disabled, and each worker repeatedly establishes a fresh SSH connection, completes authentication, executes `true`, and closes the connection. For each worker concurrency $N \in \{1, 8, 16, 32, 64\}$, we perform five runs with a 5-second warm-up, a 30-second measurement window, and a 10-second

cooldown. Throughput and latency are derived from completed connections, while CPU time and aggregate peak memory are collected from `cpu.stat` and `memory.peak`. We report the median result across the five runs.

TABLE III
CONCURRENT SSH THROUGHPUT AND SERVER CPU COST

N	Authentication Method	Successful Throughput (conn/s)	Server CPU Utilization (%)	CPU per Connection (ms)
1	KEMUAuth	11.36	11.2	19.67
1	ML-DSA-65	10.87	10.7	19.70
8	KEMUAuth	84.48	74.4	17.57
8	ML-DSA-65	81.75	72.5	17.72
16	KEMUAuth	107.99	97.6	18.06
16	ML-DSA-65	106.46	97.0	18.22
32	KEMUAuth	107.65	99.5	18.49
32	ML-DSA-65	106.48	99.5	18.69
64	KEMUAuth	106.27	99.7	18.97
64	ML-DSA-65	105.90	99.8	19.06

Table III shows that KEMUAuth achieves slightly higher successful throughput and lower CPU time per completed connection across the tested concurrency levels. At $N = 1$ and $N = 8$, its connection rate is 4.5% and 3.3% higher than that of ML-DSA-65, respectively. This represents a modest advantage while spare server capacity remains available. As the CPU approaches saturation, the throughput difference decreases to 1.4%, 1.1%, and 0.35% at $N = 16$, $N = 32$, and $N = 64$, respectively. The two methods therefore provide closely matched saturated throughput. This narrowing is expected because a complete SSH connection also includes transport key exchange, server authentication, process management, packet processing, and scheduling, which dilute the primitive-level difference. The limited change in throughput from $N = 32$ to $N = 64$ indicates that the server has reached its processing capacity, so additional concurrency primarily increases queueing rather than the connection completion rate. KEMUAuth nevertheless retains slightly lower CPU time per completed connection throughout the experiment, while the two methods remain closely matched under saturated load.

Table IV shows that the 95th-percentile latency increases as connections queue behind the saturated CPU, rising from 79–83 ms at $N = 1$ to 665–680 ms at $N = 64$. KEMUAuth retains a slightly lower tail latency at every tested concurrency level, while both methods follow nearly identical scaling trends. Aggregate peak memory also remains closely matched and reaches 242.9 MB for KEMUAuth and 244.4 MB for ML-DSA-65 at $N = 64$. Since this memory is dominated by the OpenSSH processes and common per-connection state, the additional KEM challenge state produces no visible aggregate-memory penalty in this experiment. Its protocol-specific size and lifecycle are evaluated separately in the pending-state experiment.

To isolate the memory introduced by pending KEM chal-

TABLE IV
TAIL LATENCY AND AGGREGATE MEMORY UNDER CONCURRENT LOAD

N	Authentication Method	95 th Latency (ms)	Peak Memory (MB)
1	KEMUAuth	79	9.3
1	ML-DSA-65	83	9.3
8	KEMUAuth	94	39.6
8	ML-DSA-65	97	40.1
16	KEMUAuth	158	74.4
16	ML-DSA-65	161	74.6
32	KEMUAuth	317	133.3
32	ML-DSA-65	321	133.5
64	KEMUAuth	665	242.9
64	ML-DSA-65	680	244.4

TABLE V
ESTIMATED MEMORY SCALING UNDER PENDING AUTHENTICATION STATE

Pending Connections P	Estimated Peak Memory KEMUAuth (MiB)	Estimated Peak Memory ML-DSA-65 (MiB)	Memory Difference (MiB)	Explicit KEM State (MiB)
16	74.28	70.06	4.22	0.036
32	124.03	119.86	4.17	0.073
64	223.51	219.45	4.05	0.146
128	422.48	418.65	3.83	0.291
172	559.27	555.59	3.67	0.391

lenges, we instrument challenge creation and removal and delay the client response so that multiple authentication attempts remain simultaneously pending. The ML-DSA-65 baseline is paused at the corresponding public-key authentication stage. Because independent runs occasionally reach slightly different maximum paused counts, Table V uses separate linear fits to normalize both methods to the same pending count P , rather than directly comparing unmatched raw memory peaks. The fitted per-connection growth is approximately 3.11 MiB for both methods, showing that aggregate memory is dominated by the ordinary OpenSSH process and protocol state. KEMUAuth has an estimated aggregate offset of about 4 MiB, while its explicitly retained challenge context is only 2384 bytes per pending connection and remains below 0.4 MiB at the largest observed count of 172. Pending entries return to zero after authentication completion or connection termination. These results show that KEMUAuth adds a small and bounded protocol-specific state without materially changing concurrent memory scaling.

APPENDIX D DETAILED EVALUATION RESULTS

This appendix provides the numerical results underlying Section V, following the setup in Section V-B. Tables VI and VII report sensitivity to configured RTT and random packet loss, respectively. Table VIII gives the raw data for Figure 3; and Tables IX and X provide the values underlying Figures 4 and 5, respectively.

TABLE VI
END-TO-END SSH HANDSHAKE LATENCY ACROSS CONFIGURED RTTs

RTT (ms)	KEMUAuth		ML-DSA-65		Δ_{50} (ms)
	50 th	95 th	50 th	95 th	
0	74.23	76.59	77.57	79.15	3.34
20	329.97	331.81	331.51	333.88	1.54
40	550.86	553.20	552.14	555.19	1.28
60	770.96	772.55	772.16	773.85	1.20
80	990.83	993.33	992.01	994.50	1.18
100	1211.03	1212.57	1211.67	1214.08	0.64
120	1431.31	1433.31	1432.28	1434.27	0.97
160	1870.96	1872.57	1872.24	1874.26	1.28
200	2310.94	2313.86	2312.00	2314.49	1.06

TABLE VII
END-TO-END SSH HANDSHAKE LATENCY UNDER RANDOM PACKET LOSS

Loss (%)	Authentication Method	50 th (ms)	95 th (ms)	99 th (ms)	TCP Retx. per Conn.
0	KEMUAuth	840.69	842.58	843.60	0.00
0	ML-DSA-65	841.68	843.63	844.63	0.00
0.1	KEMUAuth	840.51	842.81	1115.33	0.07
0.1	ML-DSA-65	841.49	844.05	1119.90	0.09
0.5	KEMUAuth	840.74	1116.64	1386.77	0.35
0.5	ML-DSA-65	841.71	1116.99	1844.37	0.35
1.0	KEMUAuth	841.00	1184.91	1865.98	0.75
1.0	ML-DSA-65	841.93	1123.85	1876.07	0.78
2.0	KEMUAuth	841.83	1774.46	2170.76	1.52
2.0	ML-DSA-65	842.77	1676.36	2140.88	1.52

TABLE VIII
RAW END-TO-END SSH HANDSHAKE LATENCY WITH DIFFERENT
CLIENT-AUTHENTICATION ALGORITHMS

Client Algorithm	Notation	NIST PQ Category	Mean (ms)	50 th (ms)	95 th (ms)
Ed25519	ed25519	≈0 bits	842.042	841.962	845.765
ML-DSA-44	md44	Level 2	841.720	841.622	844.368
ML-DSA-65	md65	Level 3	841.792	841.692	845.367
ML-DSA-87	md87	Level 5	841.912	841.791	845.699
Falcon-512	falcon512	Level 1	841.746	841.656	845.233
Falcon-1024	falcon1024	Level 5	841.940	841.850	846.634
SLH-DSA-SHA2-128f	sd128f	Level 1	920.703	920.556	928.641
SLH-DSA-SHA2-192f	sd192f	Level 3	929.522	929.319	934.724
SLH-DSA-SHA2-256f	sd256f	Level 5	1018.098	1017.955	1025.191
ML-KEM-512	mk512	Level 1	840.767	840.666	843.290
ML-KEM-768	mk768	Level 3	840.779	840.681	843.480
ML-KEM-1024	mk1024	Level 5	840.805	840.694	843.578
Password (yescrypt)	password	–	785.648	785.446	790.494

TABLE IX
RAW END-TO-END SSH HANDSHAKE LATENCY UNDER DIFFERENT RTT REGIMES

Client Algorithm	Server Algorithm	Notation	Close RTT = 37 ms			Intermediate RTT = 67 ms			Long RTT = 163 ms		
			Mean (ms)	50 th (ms)	95 th (ms)	Mean (ms)	50 th (ms)	95 th (ms)	Mean (ms)	50 th (ms)	95 th (ms)
ed25519	ed25519	ed25519	522.01	522.04	525.07	853.14	852.54	858.96	1911.21	1912.88	1918.35
md65 + ed25519	md65 + ed25519	h-md65	524.53	524.67	527.08	855.31	854.60	861.19	1913.69	1915.50	1920.41
sd192f + ed25519	sd192f + ed25519	h-sd192f	644.52	645.29	649.93	1033.83	1033.59	1042.34	2283.98	2285.79	2292.09
mk768 + ed25519	md65 + ed25519	h-mk768-md65	523.49	523.68	526.06	852.82	852.37	858.57	1913.37	1915.46	1920.11
mk768 + ed25519	sd192f + ed25519	h-mk768-sd192f	581.58	582.00	585.53	946.05	944.51	953.30	2096.68	2098.43	2103.70
ed25519 → md65	md65 + ed25519	ms-md65	603.55	603.85	606.64	997.94	992.99	1000.43	2247.47	2249.28	2254.37
ed25519 → mk768	md65 + ed25519	ms-mk768	600.16	600.32	603.14	990.03	988.66	996.58	2244.87	2246.80	2251.31
mk768	md65	mk768-md65	518.80	518.84	521.24	849.51	848.75	854.14	1910.46	1911.72	1916.49

TABLE X
RAW SSH HANDSHAKE LATENCY UNDER DIFFERENT TCP INITIAL CONGESTION-WINDOW SIZES

initcwnd (MSS)	A: ed25519 + ed25519			B: md65 + md65			C: sd192f + sd192f			D: mk768 + md65			E: mk768 + sd192f		
	5th	50th	95th	5th	50th	95th	5th	50th	95th	5th	50th	95th	5th	50th	95th
3	861.92	863.91	870.43	995.07	997.38	1002.71	1236.59	1238.92	1246.12	925.17	927.34	932.20	1014.53	1016.93	1021.97
5	862.84	867.87	871.95	924.08	925.85	931.75	1167.80	1172.74	1178.35	856.51	858.14	863.13	1012.55	1014.71	1021.00
7	862.07	867.14	871.34	855.44	858.63	864.56	1169.28	1173.71	1179.30	856.67	861.01	865.06	1012.38	1017.61	1021.15
10	859.23	864.15	868.61	860.18	864.78	867.94	1036.09	1042.69	1047.26	859.44	863.44	867.21	945.45	950.18	955.06
15	862.22	866.97	871.07	858.09	862.08	865.26	1038.97	1044.33	1048.63	856.95	859.97	863.45	946.43	951.21	955.22
20	860.24	864.87	868.92	856.57	861.04	864.85	1036.98	1043.01	1047.72	857.35	861.69	865.77	944.99	950.44	954.85
25	859.52	864.93	869.72	857.50	862.15	866.56	968.77	975.13	980.04	859.12	864.28	867.88	948.44	953.63	958.28
30	861.46	864.13	870.20	858.35	860.82	866.06	904.29	907.19	912.07	859.54	862.40	866.26	879.94	883.03	887.58
35	859.94	863.49	869.28	859.60	862.38	867.48	904.52	908.02	912.84	858.80	861.37	866.01	881.66	884.58	888.73
40	859.60	861.44	867.76	857.23	859.22	864.47	901.47	904.88	910.10	856.06	858.28	863.77	879.60	882.63	888.24
50	861.10	864.77	870.32	860.71	862.17	867.35	904.15	906.76	911.24	858.97	862.48	866.57	879.59	883.04	887.17